

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



**DỰ ĐOÁN GIÁ XE Ô TÔ CŨ CÓ XÉT ĐẾN TÌNH
TRẠNG HƯ HỎNG NGOẠI THẤT BẰNG
MACHINE LEARNING VÀ COMPUTER VISION**

Môn học: Thực tập cơ sở

Nguyễn Đức Sinh B23DCKH098

Giảng viên hướng dẫn: TS. Nguyễn Xuân Đức

Hà Nội – 2026

Mục lục

1. Tổng quan đề tài	4
1.1. Giới thiệu bài toán định giá xe cũ	4
1.2. Lý do chọn đề tài	4
1.3. Mục tiêu dự án	5
1.4. Phạm vi	5
2. Cơ sở lý thuyết	6
2.1. Bài toán hồi quy trong Machine Learning	6
2.2. Random Forest, Linear Regression, XGBoost	6
2.2.1 Random Forest	6
2.2.2 Linear Regression	8
2.2.3 XGBoost	10
2.3. YOLO cho object detection	12
2.4. CNN/ConvNeXt-Tiny cho phân loại mức độ hư hỏng	16
2.5. Các metric: RMSE, MAE, R^2 , mAP, accuracy	16
2.5.1. Metrics Cho Bài Toán Dự Đoán Giá Xe	17
2.5.2 Metrics Cho Bài Toán Object Detection Bằng YOLO	17
2.5.3. Metrics cho bài toán phân loại mức độ hư hỏng	18
2.5.4. Vai trò của metrics trong dự án	18
3. Dataset và tiền xử lý	19
3.1. Dataset giá xe cũ dạng bảng	19
3.1.1. Quy mô và cấu trúc tổng quát của dataset	19
3.1.2. Ý nghĩa của các cột dữ liệu	21
3.2. Dataset phát hiện hư hỏng xe	23
3.2.1. Nguồn dữ liệu	23
3.2.2. Phân bố nhãn hư hỏng	25
3.2.3. Đặc điểm thị giác của từng lớp hư hỏng	26
3.2.4. Độ đa dạng về góc chụp và điều kiện ánh	30
3.3. Dataset phân loại mức độ hư hỏng	35
3.3.1. Nguồn dữ liệu và mục đích sử dụng	35

3.3.2. Tiêu chí phân loại mức độ hư hỏng	35
3.3.2. Đặc điểm và chất lượng hình ảnh	38
3.4. Nhận xét về dữ liệu	41
4. Phương pháp xây dựng mô hình	43
4.1. Mô hình dự đoán giá từ dữ liệu bảng.....	44
4.2. Mô hình phát hiện hư hỏng bằng YOLOv8	44
4.3. Mô hình phân loại mức độ hư hỏng bằng ConvNeXt-Tiny.....	45
4.4. Cơ chế điều chỉnh giá sau hư hỏng	46
4.5. Hiển Thị Kết Quả Và Khả Năng Giải Thích Trên Giao Diện	48
5. Thiết kế hệ thống.....	51
5.1. Kiến trúc tổng thể.....	51
5.2. Luồng xử lý	53
5.3. Các module chính.....	55
6. Kết quả thực nghiệm	57
6.1. Kết quả mô hình dự đoán giá từ dữ liệu bảng.....	57
6.2. Kết quả thực nghiệm mô hình YOLO.....	59
6.2.1. Kết quả huấn luyện trên các cấu hình nền tảng	59
6.2.2. Tác động của dataset merge_data	59
6.2.3. Quyết định lựa chọn mô hình cốt lõi merge_data.....	60
6.2.4. Nhận xét so sánh với Faster R-CNN.....	66
6.3. Kết quả CNN.....	66
7. Đánh giá, hạn chế và hướng phát triển.....	73
7.1. Hạn chế.....	73
7.2. Hướng phát triển	74
8. Kết luận	75
9. Tài liệu tham khảo.....	77

LỜI MỞ ĐẦU

Lời đầu tiên, em xin gửi lời cảm ơn chân thành đến Học viện Công nghệ Bưu chính Viễn thông đã tạo điều kiện cho em được tiếp cận với môn học Thực tập cơ sở – một học phần quan trọng giúp em bước đầu làm quen với quy trình xây dựng một sản phẩm công nghệ thực tế.

Đặc biệt, em xin gửi lời cảm ơn sâu sắc nhất tới thầy **Nguyễn Xuân Đức**. Trong suốt quá trình thực hiện học phần này, thầy đã tận tình hướng dẫn, định hướng chuyên môn và cung cấp cho em những kiến thức nền tảng quý báu để em có thể hoàn thành đề tài dự án cá nhân của mình.

Hiện nay, cùng với sự phát triển mạnh mẽ của thị trường xe hơi, việc xác định giá trị thực của một chiếc xe đã qua sử dụng trở thành nhu cầu thiết yếu của cả người mua lẫn người bán. Nhận thấy tiềm năng và tính ứng dụng thực tiễn của trí tuệ nhân tạo trong việc giải quyết bài toán này, em đã quyết định lựa chọn đề tài: **“Dự đoán giá xe ô tô cũ có xét đến tình trạng hư hỏng ngoại thất bằng Machine Learning và Computer Vision”**. Đề tài hướng đến việc kết hợp dữ liệu dạng bảng với dữ liệu ảnh ngoại thất để hỗ trợ ước lượng giá xe cũ một cách trực quan và gần với tình huống thực tế hơn.

Do đây là lần đầu tiên trực tiếp triển khai một dự án Machine Learning và Computer Vision từ bước thu thập dữ liệu, tiền xử lý, huấn luyện mô hình cho đến tích hợp vào ứng dụng demo, mặc dù đã nỗ lực hết mình nhưng bài báo cáo chắc chắn không tránh khỏi những thiếu sót về mặt kỹ thuật hay cách trình bày. Em rất mong nhận được những ý kiến đóng góp, nhận xét từ thầy để em có thể rút kinh nghiệm và hoàn thiện tư duy lập trình cũng như kiến thức chuyên môn của mình trong tương lai.

Cuối cùng, em xin kính chúc thầy thật nhiều sức khỏe, hạnh phúc và gặt hái được nhiều thành công hơn nữa trong sự nghiệp giảng dạy.

Em xin chân thành cảm ơn!

1. Tổng quan đề tài

1.1. Giới thiệu bài toán định giá xe cũ

Trong kỷ nguyên công nghệ 4.0, sự bùng nổ của dữ liệu và sức mạnh tính toán đã đưa Học máy (Machine Learning) trở thành công cụ cốt lõi trong việc ra quyết định ở nhiều lĩnh vực. Trong đó, thị trường ô tô đã qua sử dụng (Used Cars) là một thị trường có tính biến động cao và chịu ảnh hưởng bởi rất nhiều yếu tố đan xen.

Việc xác định giá trị thực của một chiếc xe cũ từ trước đến nay thường dựa vào kinh nghiệm cá nhân của các chuyên gia hoặc sự thỏa thuận cảm tính giữa người mua và người bán. Song điều này dẫn đến sự bất cân xứng thông tin, gây khó khăn cho việc định giá nhanh và chính xác dựa trên quy mô lớn. Một chiếc xe không chỉ mất giá theo thời gian mà còn phụ thuộc vào:

- Thông số kỹ thuật: Loại động cơ, công suất, loại nhiên liệu sử dụng
- Lịch sử vận hành: Số km đã đi, số đời chủ sở hữu

Chính vì vậy, việc xây dựng một mô hình hỗ trợ dự báo giá xe tự động không chỉ giúp tối ưu hóa quy trình tham khảo giá mà còn góp phần tăng tính minh bạch cho thị trường. Đề tài này được thực hiện nhằm nghiên cứu khả năng áp dụng các thuật toán hồi quy (Regression) để xử lý dữ liệu thực tế từ thị trường ô tô cũ, từ đó xây dựng quy trình cơ bản cho bài toán định giá.

1.2. Lý do chọn đề tài

Trong những năm gần đây, thị trường xe ô tô cũ ngày càng phát triển do nhu cầu mua bán, trao đổi phương tiện đã qua sử dụng tăng cao. Tuy nhiên, việc xác định giá trị thực tế của một chiếc xe cũ vẫn còn phụ thuộc nhiều vào kinh nghiệm cá nhân, cảm tính của người mua bán hoặc đánh giá thủ công từ các đơn vị trung gian. Giá xe cũ chịu ảnh hưởng bởi nhiều yếu tố như năm sản xuất, số km đã đi, hãng xe, loại nhiên liệu, hộp số, số chủ sở hữu và đặc biệt là tình trạng ngoại thất của xe.

Trong thực tế, hai chiếc xe có cùng thông số kỹ thuật nhưng tình trạng hư hỏng khác nhau có thể có giá trị chênh lệch đáng kể. Vì vậy, nếu chỉ dự đoán giá dựa trên dữ liệu dạng bảng mà không xét đến tình trạng hư hỏng từ hình ảnh thì kết quả định giá có thể chưa phản ánh đầy đủ giá trị thực của xe.

Xuất phát từ vấn đề đó, em lựa chọn đề tài “Dự đoán giá xe ô tô cũ có xét đến tình trạng hư hỏng ngoại thất bằng Machine Learning và Computer Vision”. Đề tài

kết hợp giữa học máy trên dữ liệu bảng và thị giác máy tính trên dữ liệu ảnh, giúp hệ thống không chỉ dự đoán giá cơ bản của xe mà còn phát hiện hư hỏng, đánh giá mức độ nghiêm trọng và đưa ra giá tham khảo sau điều chỉnh.

Đề tài cũng phù hợp với nội dung môn Thực tập cơ sở vì giúp em vận dụng nhiều kiến thức đã học như xử lý dữ liệu, huấn luyện mô hình Machine Learning, sử dụng mô hình học sâu, xây dựng pipeline xử lý và triển khai ứng dụng demo bằng Streamlit.

1.3. Mục tiêu dự án

Mục tiêu chính của dự án là xây dựng một hệ thống demo có khả năng hỗ trợ định giá xe ô tô cũ dựa trên thông tin kỹ thuật của xe và hình ảnh ngoại thất. Hệ thống hướng đến việc tạo ra một quy trình định giá tương đối hoàn chỉnh, bao gồm dự đoán giá cơ bản, phát hiện hư hỏng, phân loại mức độ hư hỏng và đưa ra giá tham khảo sau điều chỉnh.

Các mục tiêu cụ thể của dự án gồm:

- Thu thập, phân tích và tiền xử lý dữ liệu xe ô tô cũ dạng bảng để phục vụ bài toán dự đoán giá.
- Xây dựng và đánh giá các mô hình hồi quy như Linear Regression, Random Forest và XGBoost để dự đoán giá cơ bản của xe.
- Tích hợp mô hình YOLO để phát hiện các vùng hư hỏng trên ảnh xe.
- Sử dụng mô hình CNN/ConvNeXt-Tiny để phân loại mức độ hư hỏng tổng quát từ ảnh full người dùng upload thành các mức minor, moderate và severe.
- Xây dựng cơ chế điều chỉnh giá tham khảo dựa trên số lượng hư hỏng, loại hư hỏng, diện tích vùng hỏng và mức độ nghiêm trọng.
- Phát triển giao diện demo bằng Streamlit, cho phép người dùng nhập thông tin xe, tải ảnh lên và nhận kết quả dự đoán trực quan.
- Đánh giá ưu điểm, hạn chế của hệ thống và đề xuất hướng phát triển trong tương lai.

Thông qua dự án này, em mong muốn hiểu rõ hơn quy trình xây dựng một sản phẩm Machine Learning hoàn chỉnh, từ bước chuẩn bị dữ liệu, huấn luyện mô hình, đánh giá kết quả cho đến tích hợp thành một ứng dụng có thể sử dụng được.

1.4. Phạm vi

Dự án được giới hạn trong phạm vi xây dựng một hệ thống demo hỗ trợ ước lượng giá xe ô tô cũ, kết hợp dữ liệu bảng và dữ liệu ảnh ngoại thất. Hệ thống chưa hướng đến triển khai production hoặc thay thế hoàn toàn quy trình thẩm định xe thực tế.

Phạm vi thực hiện:

- Dự đoán giá cơ bản của xe dựa trên dữ liệu dạng bảng gồm các thuộc tính kỹ thuật và lịch sử sử dụng.
- Phát hiện một số dạng hư hỏng ngoại thất phổ biến trên ảnh xe bằng YOLOv8.
- Phân loại mức độ hư hỏng thành các mức nhẹ, trung bình và nghiêm trọng bằng CNN/ConvNeXt-Tiny.
- Tính giá tham khảo sau điều chỉnh bằng cơ chế rule-based dựa trên kết quả phát hiện của YOLOv8s và mức độ severity từ ConvNeXt-Tiny.
- Xây dựng giao diện demo bằng Streamlit để minh họa luồng nhập thông tin xe, tải ảnh và hiển thị kết quả.

Giới hạn của phạm vi:

- Các bộ dữ liệu bảng và ảnh chưa đồng bộ hoàn toàn theo từng chiếc xe.
- Dự án chưa có nhãn ground truth cho giá xe sau khi xét đến từng loại hư hỏng cụ thể.
- Giá tham khảo sau điều chỉnh mang tính minh họa cho ý tưởng kết hợp Computer Vision vào bài toán định giá, chưa đại diện cho giá giao dịch thực tế.

Công nghệ sử dụng:

- Ngôn ngữ lập trình: Python.
- Xử lý dữ liệu và trực quan hóa: Pandas, NumPy, Matplotlib, Seaborn.
- Machine Learning: Scikit-learn, XGBoost.
- Deep Learning và Computer Vision: PyTorch, ConvNeXt-Tiny, YOLOv8/Ultralytics, OpenCV/PIL.
- Giao diện demo: Streamlit.

2. Cơ sở lý thuyết

2.1. Bài toán hồi quy trong Machine Learning

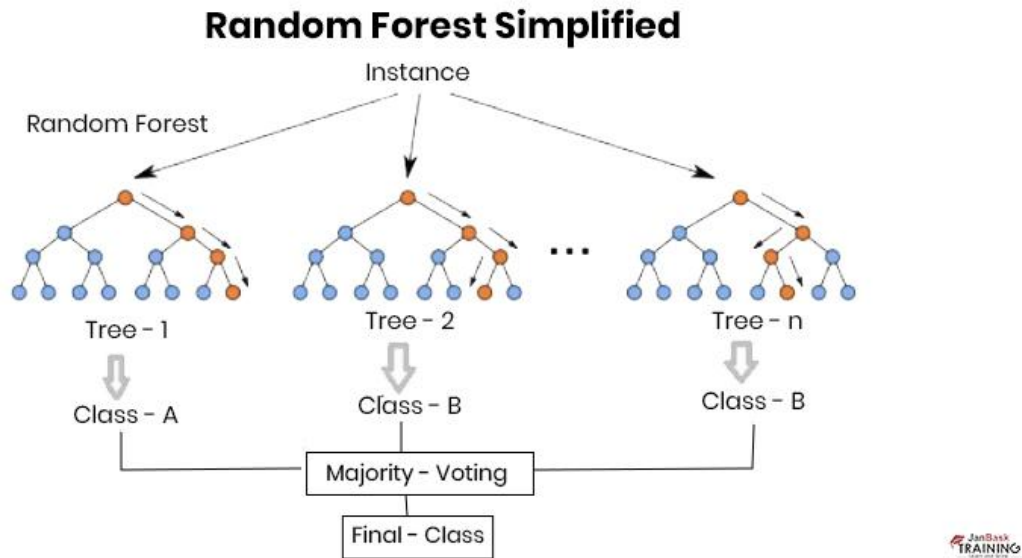
2.2. Random Forest, Linear Regression, XGBoost

2.2.1 Random Forest

2.2.1.1. Khái niệm

Random Forest là một thuật toán học có giám sát, thuộc nhóm các thuật toán ensemble learning. Thuật toán này hoạt động bằng cách xây dựng nhiều cây quyết định(

Decision Tree) trong quá trình đào tạo và đưa ra dự đoán bằng cách tính trung bình các dự đoán của các cây quyết định đó hoặc chọn lớp có số phiếu bầu nhiều nhất trong trường hợp phân loại. Mỗi cây trong Random Forest được xây dựng từ một mẫu có thể của dữ liệu đào tạo (bootstrapping) và trong quá trình xây dựng mỗi cây, chỉ một tập con ngẫu nhiên của các đặc điểm được sử dụng để chia các nút của cây.



Hình 1: Mô hình Random Forest

2.2.1.2. Cách thức hoạt động

- Bước 1: Chọn điểm dữ liệu K ngẫu nhiên từ tập huấn luyện.
- Bước 2: Xây dựng cây quyết định liên kết với các điểm dữ liệu đã chọn (Tập con).
- Bước 3: Chọn số N cho cây quyết định mà bạn muốn xây dựng.
- Bước 4: Lặp lại Bước 1 và 2.
- Bước 5: Đối với các điểm dữ liệu mới, hãy tìm các dự đoán của từng cây quyết định và gán các điểm dữ liệu mới cho danh mục giành được đa số phiếu bầu.

2.2.1.3. Ưu và nhược điểm của thuật toán

- Ưu điểm:

- **Hiệu quả cao:** Random Forest có khả năng cung cấp kết quả chính xác cao, ngay cả trên các bộ dữ liệu lớn và phức tạp.
- **Kiểm soát overfitting:** Mặc dù từng cây quyết định có thể bị overfit, kết hợp kết quả từ nhiều cây giúp giảm thiểu vấn đề này.
- **Dễ dàng đo lường mức độ quan trọng của từng đặc điểm:** Random Forest có thể cung cấp thông tin chi tiết về mức độ quan trọng của các đặc điểm đầu vào đối với quá trình dự đoán.

- **Khả năng xử lý dữ liệu cả bảng và dạng phân loại:** Thuật toán này có thể xử lý cả các biến định lượng và định tính.
- **Nhược điểm:**
 - **Không trực quan và khó giải thích:** Mô hình Random Forest gồm nhiều cây quyết định có thể khó hiểu và giải thích so với một cây quyết định đơn lẻ.
 - **Thời gian đào tạo có thể lâu hơn:** Xây dựng nhiều cây đòi hỏi nhiều tài nguyên tính toán hơn, đặc biệt là trên các bộ dữ liệu lớn.
 - **Độ phức tạp cao khi triển khai:** Khi số lượng cây trong rừng tăng lên, mô hình cần nhiều bộ nhớ hơn và thời gian phản hồi có thể chậm hơn.

2.2.1.4. Vai trò của Random Forest trong dự án

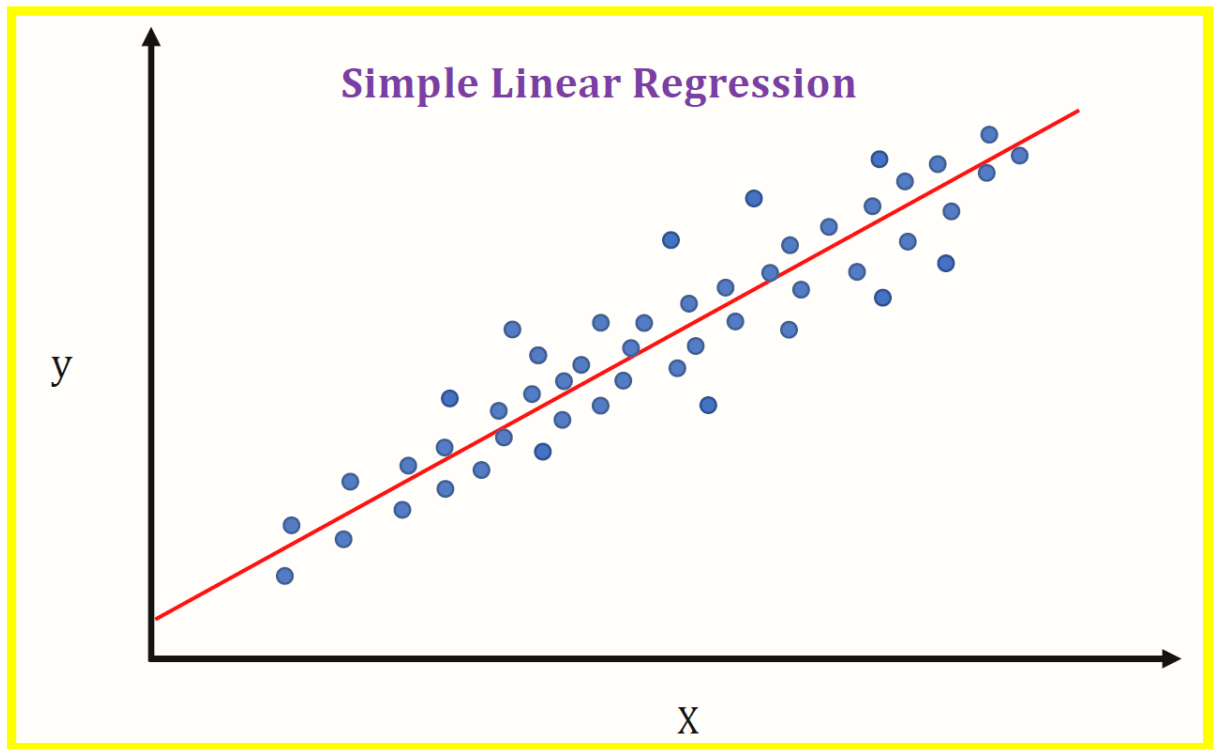
Trong dự án, Random Forest được sử dụng như một mô hình hồi quy phi tuyến để so sánh với Linear Regression và XGBoost trên dữ liệu giá xe cũ. Thuật toán này phù hợp với dữ liệu bảng vì có thể học các quan hệ phức tạp giữa tuổi xe, số km đã đi, công suất, dung tích động cơ, loại nhiên liệu và giá bán.

Ngoài vai trò dự đoán, Random Forest còn giúp tham khảo mức độ quan trọng của các đặc trưng đầu vào. Tuy nhiên, do mô hình gồm nhiều cây quyết định nên khả năng giải thích chi tiết từng dự đoán không trực quan bằng mô hình tuyến tính.

2.2.2 Linear Regression

2.2.2.1. Khái niệm

Hồi quy tuyến tính (Linear Regression) là một thuật toán học máy có giám sát được sử dụng để dự đoán giá trị của một biến mục tiêu (biến phụ thuộc) dựa trên một hoặc nhiều biến dự báo (biến độc lập). Thuật toán này dựa trên giả định rằng có một mối quan hệ tuyến tính (đường thẳng) giữa các biến đầu vào và kết quả đầu ra. Kết quả của mô hình là một giá trị liên tục (định lượng), ví dụ như giá cả, điểm số hoặc nhiệt độ.



Hình 2: Minh họa mô hình hồi quy tuyến tính

2.2.2.2. Cách thức hoạt động

- **Tìm đường tiệm cận tốt nhất:** Thuật toán cố gắng vẽ một đường thẳng (hoặc siêu phẳng trong không gian nhiều chiều) đi qua các điểm dữ liệu sao cho tổng sai số giữa giá trị dự đoán và giá trị thực tế là nhỏ nhất.
- **Phương pháp Bình phương tối thiểu (OLS):** Đây là kỹ thuật phổ biến nhất để tìm ra các hệ số của phương pháp hồi quy bằng cách tối thiểu hóa tổng bình phương của các khoảng cách sai lệch (phần dư).
- **Hệ số hồi quy:** Mỗi biến đầu vào sẽ được gán một trọng số, thể hiện mức độ và chiều hướng ảnh hưởng của biến đó đến kết quả cuối cùng.

2.2.2.3. Ưu và nhược điểm của thuật toán

- **Ưu điểm:**
 - **Đơn giản và dễ giải thích:** Kết quả của mô hình rất trực quan, giúp người dùng hiểu rõ biến nào quan trọng và ảnh hưởng như thế nào đến kết quả.
 - **Hiệu suất tính toán cao:** Thuật toán chạy rất nhanh, tốn ít tài nguyên và dễ dàng triển khai trên các hệ thống thực tế.
 - **Hiệu quả với dữ liệu tuyến tính:** Nếu mối quan hệ giữa các biến là tuyến tính, mô hình này thường cho kết quả rất ổn định.

- **Nhược điểm:**

- **Giới hạn về tính tuyến tính:** Thuật toán không thể học được các mối quan hệ phức tạp hoặc phi tuyến giữa các dữ liệu.
- **Nhạy cảm với ngoại lệ (Outliers):** Chỉ cần một vài điểm dữ liệu có giá trị bất thường cũng có thể làm lệch hướng đường hồi quy, dẫn đến sai số lớn.
- **Hiện tượng đa cộng tuyến:** Nếu các biến độc lập có liên quan quá chặt chẽ với nhau, mô hình có thể trở nên thiếu chính xác

2.2.2.4. Vai trò của Linear Regression trong dự án

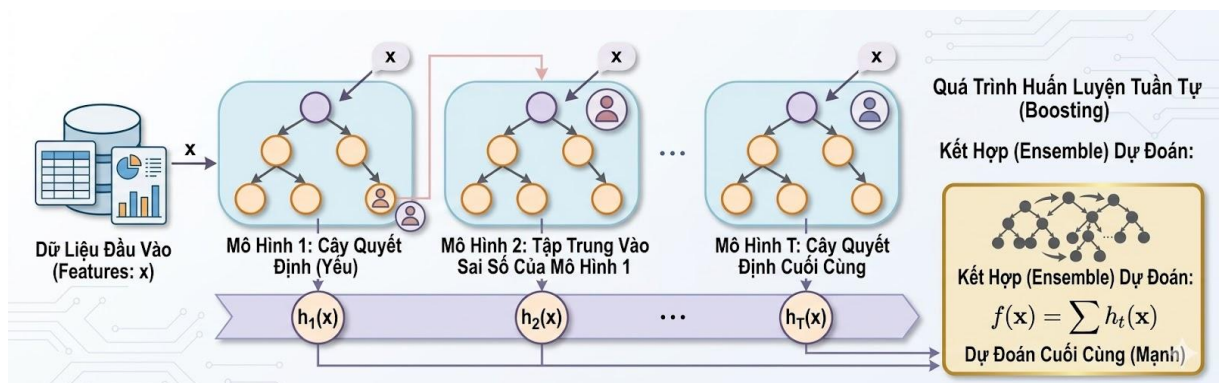
Trong dự án, Linear Regression được dùng làm mô hình baseline cho bài toán dự đoán giá xe. Mục đích chính của mô hình này là tạo ra một mốc so sánh đơn giản để đánh giá xem các mô hình phi tuyến như Random Forest và XGBoost cải thiện kết quả đến mức nào.

Do bài toán giá xe cũ chịu ảnh hưởng bởi nhiều yếu tố phi tuyến như tuổi xe, số km đã đi, hãng xe và loại nhiên liệu, Linear Regression thường khó đạt kết quả tốt nhất. Tuy vậy, mô hình này vẫn có giá trị vì dễ huấn luyện, dễ giải thích và giúp kiểm tra nhanh xu hướng của dữ liệu.

2.2.3. XGBoost

2.2.3.1. Khái niệm

XGBoost (Extreme Gradient Boosting) là một thuật toán học máy mạnh mẽ được phát triển dựa trên thuật toán Gradient Boosting Trees (Cây Boosting Gradient). XGBoost được biết đến với khả năng học tập hiệu quả, chính xác cao và có thể giải quyết được nhiều loại bài toán học máy khác nhau, đặc biệt là các bài toán phân loại và hồi quy.



Hình 3: Quy trình huấn luyện tuần tự của XGBoost

2.2.3.2. Cách thức hoạt động

- **Xây dựng tuần tự (Sequential Building):** Khác với Random Forest xây dựng các cây độc lập, XGBoost xây dựng các cây quyết định một cách tuần tự. Mỗi cây mới được thêm vào sẽ cố gắng học và sửa chữa những sai số (residuals) mà các cây trước đó đã mắc phải.
- **Tối ưu hóa Gradient:** Thuật toán sử dụng phương pháp hạ độ dốc (Gradient Descent) để giảm thiểu hàm mất mát. Điểm đặc biệt của XGBoost là sử dụng khai triển Taylor bậc hai để xấp xỉ hàm mất mát, giúp quá trình tìm ra cấu trúc cây tối ưu diễn ra nhanh và chính xác hơn.
- **Cơ chế cắt tỉa (Pruning):** XGBoost sử dụng tham số "gain" để quyết định xem một nút có nên được chia hay không. Nó xây dựng cây đến độ sâu tối đa trước, sau đó mới cắt tỉa ngược lại từ dưới lên để loại bỏ những nhánh không mang lại giá trị đáng kể cho mô hình.
- **Xử lý dữ liệu khuyết (Sparsity-aware splitting):** Thuật toán có khả năng tự động học hướng phân tách tốt nhất khi gặp các giá trị bị thiếu trong dữ liệu, giúp mô hình hoạt động ổn định mà không cần xử lý dữ liệu trống quá phức tạp

2.2.3.3. Ưu và nhược điểm của thuật toán

- **Ưu điểm:**
 - **Hiệu suất cao và nhanh chóng:** XGBoost được tối ưu hóa để hiệu quả cả về tốc độ lẫn sử dụng bộ nhớ, thường nhanh hơn các thuật toán gradient boosting truyền thống.
 - **Khả năng xử lý quy mô lớn:** Thuật toán có thể xử lý hàng triệu dữ liệu và tương thích tốt với các nền tảng tính toán song song và phân tán.
 - **Chống quá khớp:** XGBoost cung cấp các tính năng như cắt tỉa cây (pruning), điều chỉnh độ sâu của cây, và định thời (regularization), giúp tránh hiện tượng quá khớp một cách hiệu quả.
 - **Tính linh hoạt:** Có thể tùy chỉnh được nhiều tham số và hỗ trợ các hàm mục tiêu tùy chỉnh và hàm đánh giá trong quá trình huấn luyện.
 - **Xử lý các tính năng dạng số và dạng danh mục một cách tự động.**
- **Nhược điểm:**

- **Đòi hỏi kiến thức kỹ thuật:** Việc cần phải hiểu và tinh chỉnh nhiều tham số có thể làm cho XGBoost khó sử dụng hiệu quả đối với người mới bắt đầu.
- **Thời gian huấn luyện có thể lâu:** Dù được tối ưu hóa, nhưng việc xây dựng hàng trăm cây có thể tốn thời gian, đặc biệt là trên các bộ dữ liệu cực lớn.
- **Không phù hợp cho dữ liệu có chiều dữ liệu cao và thừa thớt:** Trong một số trường hợp, các mô hình tuyến tính có thể hiệu quả hơn đối với dữ liệu thưa.

2.2.3.4. Vai trò của XGBoost trong dự án

Trong dự án, XGBoost được lựa chọn là mô hình chính cho nhánh dự đoán giá cơ bản vì có khả năng xử lý tốt dữ liệu bảng, học được quan hệ phi tuyến và thường đạt hiệu quả cao trong các bài toán regression.

So với Linear Regression, XGBoost biểu diễn tốt hơn các tương tác phức tạp giữa các đặc trưng. So với Random Forest, XGBoost có cơ chế học tuần tự để sửa sai qua từng cây, nhờ đó đạt kết quả tốt hơn trong phần thực nghiệm của dự án.

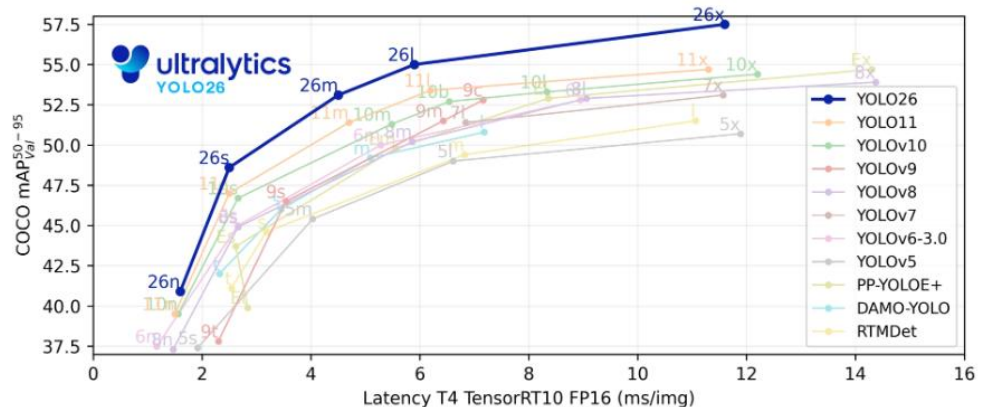
2.3. YOLO cho object detection

Trong dự án, mô hình YOLO được sử dụng cho bài toán phát hiện đối tượng, cụ thể là phát hiện các vùng hư hỏng trên ảnh xe ô tô. YOLO, viết tắt của “You Only Look Once”, là một mô hình phát hiện đối tượng thuộc nhóm one-stage detector. Khác với các phương pháp phát hiện đối tượng truyền thống cần nhiều bước xử lý, YOLO có khả năng xác định vị trí và phân loại đối tượng trong cùng một lần suy luận. Nhờ đó, mô hình có tốc độ xử lý nhanh và phù hợp với các ứng dụng cần phản hồi gần thời gian thực.

Ultralytics YOLO26

Overview

Ultralytics YOLO26 is the latest evolution in the YOLO series of real-time object detectors, engineered from the ground up for **edge and low-power devices**. It introduces a streamlined design that removes unnecessary complexity while integrating targeted innovations to deliver faster, lighter, and more accessible deployment.



[Try on Ultralytics Platform](#)

Explore and run YOLO26 models directly on [Ultralytics Platform](#).

Hình 4: Tổng quan hiệu năng của các phiên bản YOLO

Lý do lựa chọn YOLO trong dự án là vì mô hình này có sự cân bằng tốt giữa tốc độ và độ chính xác. Đối với bài toán đánh giá tình trạng xe cũ, hệ thống cần phát hiện được các hư hỏng ngoại thất như vết móp, vết xước, nứt vỡ hoặc hư hỏng đèn kính từ ảnh do người dùng cung cấp. YOLO cho phép phát hiện trực tiếp các vùng hư hỏng này thông qua bounding box, đồng thời trả về nhãn loại hư hỏng và độ tin cậy của từng dự đoán.

Trong hệ thống, YOLO đóng vai trò là mô-đun thị giác máy tính đầu tiên trong pipeline xử lý ảnh. Sau khi người dùng tải ảnh xe lên giao diện, ảnh sẽ được đưa vào mô hình YOLO để phát hiện các vùng hư hỏng. Kết quả đầu ra của mô hình bao gồm tọa độ bounding box, loại hư hỏng, confidence score và tỷ lệ diện tích vùng hư hỏng so với toàn bộ ảnh. Các thông tin này không chỉ được dùng để hiển thị trực quan trên giao diện mà còn là đầu vào cho các bước xử lý tiếp theo.

Các lớp hư hỏng được sử dụng trong dự án bao gồm một số loại phổ biến như dent, scratch, crack, lamp_broken, glass_broken và tire_flat. Dữ liệu huấn luyện được tổ chức theo định dạng YOLO, trong đó mỗi ảnh đi kèm với file nhãn chứa thông tin về class và tọa độ vùng hư hỏng. Mô hình sau khi huấn

luyện được lưu lại dưới dạng file trọng số và được tích hợp vào hệ thống để phục vụ quá trình suy luận.

Sau khi YOLO phát hiện các vùng hư hỏng, hệ thống vẽ bounding box lên ảnh để người dùng quan sát trực tiếp kết quả. Ở phiên bản hiện tại, YOLOv8s và ConvNeXt-Tiny là hai nhánh xử lý ảnh song song: YOLOv8s tạo ra các damage features như loại hư hỏng, số lượng vùng hỏng, confidence và area ratio, còn ConvNeXt-Tiny nhận ảnh full để dự đoán mức độ severity tổng quát. Các thông tin này được kết hợp ở tầng rule-based adjustment để điều chỉnh giá xe.

Nhờ việc tích hợp YOLO, hệ thống không chỉ dự đoán giá xe dựa trên dữ liệu dạng bảng mà còn có khả năng khai thác thông tin từ hình ảnh thực tế của xe. Điều này giúp kết quả định giá phản ánh tốt hơn tình trạng ngoại thất, từ đó làm cho hệ thống gần với bài toán định giá xe cũ trong thực tế hơn.

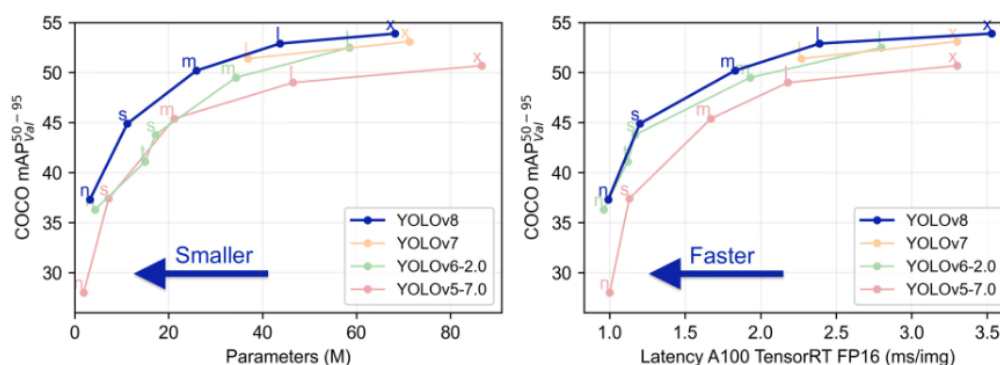
YOLOv8 hỗ trợ tốt cho bài toán phát hiện nhiều loại hư hỏng trên ảnh xe trong cùng một khung hình. Trong dự án, hệ thống cần nhận diện các vùng hư hỏng ngoại thất. Đây là các dạng hư hỏng có đặc điểm thị giác khác nhau: vết xước thường nhỏ và mảnh, vết móp có thể biến dạng theo bề mặt thân xe, trong khi kính hoặc đèn vỡ thường có hình dạng rõ hơn. YOLOv8 cho phép học đồng thời nhiều lớp hư hỏng như vậy, đồng thời trả về vị trí bounding box, nhãn hư hỏng và độ tin cậy của từng dự đoán. Điều này phù hợp với yêu cầu của hệ thống vì kết quả phát hiện không chỉ dùng để hiển thị trực quan mà còn được đưa sang các bước phân loại mức độ hư hỏng và điều chỉnh giá xe.



Explore Ultralytics YOLOv8

Overview

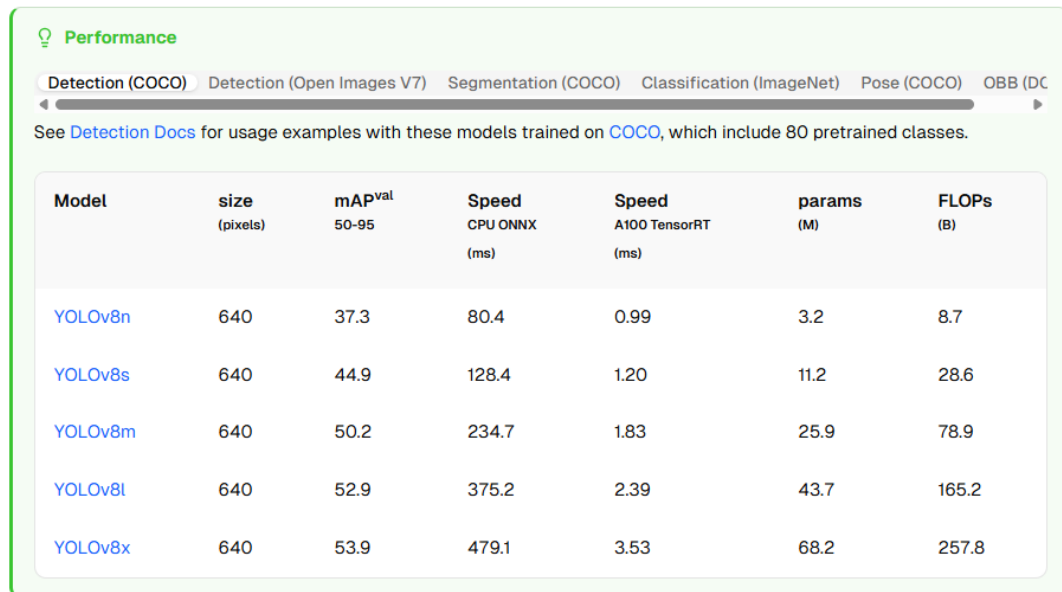
YOLOv8 was released by Ultralytics on January 10, 2023, offering cutting-edge performance in terms of accuracy and speed. Building upon the advancements of previous YOLO versions, YOLOv8 introduced new features and optimizations that make it an ideal choice for various [object detection](#) tasks in a wide range of applications.



Hình 5: So sánh hiệu năng và độ trễ của YOLOv8

YOLOv8 có nhiều biến thể kích thước khác nhau như YOLOv8n, YOLOv8s và YOLOv8m, cho phép thử nghiệm và đánh đổi giữa tốc độ với độ chính xác. Trong quá trình huấn luyện, dự án đã so sánh nhiều biến thể YOLOv8 trên bộ dữ liệu phát hiện hư hỏng.

- YOLOv8n có ưu điểm nhẹ và suy luận nhanh nhưng độ chính xác tổng thể chưa phải tốt nhất.
- YOLOv8m có khả năng học mạnh hơn nhưng kích thước mô hình và chi phí tính toán cao hơn, không thật sự phù hợp với mục tiêu tích hợp vào ứng dụng demo.
- YOLOv8s cho thấy mức cân bằng tốt hơn giữa độ chính xác, tốc độ xử lý và khả năng triển khai. Sau quá trình so sánh, checkpoint tốt nhất của cấu hình YOLOv8s trên merge_data được chọn làm mô hình phát hiện hư hỏng chính của hệ thống. Việc lựa chọn YOLOv8s phù hợp với định hướng nghiên cứu và triển khai của dự án.



The screenshot shows the 'Performance' page of the YOLOv8 repository. It features a navigation bar with tabs for different tasks: Detection (COCO), Detection (Open Images V7), Segmentation (COCO), Classification (ImageNet), Pose (COCO), and OBB (DC). The 'Detection (COCO)' tab is selected. Below the navigation bar, there is a link to 'Detection Docs' and a note about 80 pretrained classes. The main content is a table with the following columns: Model, size (pixels), mAP^{val} 50-95, Speed CPU ONNX (ms), Speed A100 TensorRT (ms), params (M), and FLOPs (B). The table lists five models: YOLOv8n, YOLOv8s, YOLOv8m, YOLOv8l, and YOLOv8x, with their respective metrics.

Model	size (pixels)	mAP ^{val} 50-95	Speed CPU ONNX (ms)	Speed A100 TensorRT (ms)	params (M)	FLOPs (B)
YOLOv8n	640	37.3	80.4	0.99	3.2	8.7
YOLOv8s	640	44.9	128.4	1.20	11.2	28.6
YOLOv8m	640	50.2	234.7	1.83	25.9	78.9
YOLOv8l	640	52.9	375.2	2.39	43.7	165.2
YOLOv8x	640	53.9	479.1	3.53	68.2	257.8

Hình 6: Bảng thông số hiệu năng các biến thể YOLOv8

Thông qua mô hình này, dự án có thể tiếp cận đầy đủ quy trình của một bài toán Computer Vision hiện đại, từ chuẩn bị dữ liệu ảnh, chuyển đổi annotation sang định dạng YOLO, huấn luyện mô hình, đánh giá bằng các chỉ số như **Precision**, **Recall**, **mAP**, đến tích hợp mô hình vào pipeline hoàn chỉnh.

Việc tìm hiểu về YOLOv8 coi như là bước đầu cho việc tìm hiểu sâu hơn về Computer Vision.

2.4. CNN/ConvNeXt-Tiny cho phân loại mức độ hư hỏng

CNN được sử dụng trong dự án để học đặc trưng thị giác từ ảnh xe và phân loại mức độ hư hỏng tổng quát của ảnh thành ba mức minor, moderate và severe, tương ứng với hư hỏng nhẹ, trung bình và nghiêm trọng. Khác với thiết kế cũ, mô hình CNN hiện tại không nhận ảnh cắt từ bounding box của YOLO mà nhận trực tiếp ảnh full do người dùng upload.

Trong pipeline mới, YOLOv8s và CNN/ConvNeXt-Tiny được xem là hai nhánh xử lý ảnh song song. YOLOv8s tập trung phát hiện vị trí và loại hư hỏng, còn ConvNeXt-Tiny đánh giá mức độ hư hỏng tổng quát của toàn bộ ảnh. Cách thiết kế này giúp hệ thống tận dụng cả thông tin cục bộ từ YOLO và thông tin ngữ cảnh tổng thể từ ảnh full.

Dự án lựa chọn ConvNeXt-Tiny làm kiến trúc CNN chính. ConvNeXt-Tiny là một kiến trúc CNN hiện đại, kế thừa ưu điểm của CNN truyền thống nhưng được cải tiến theo hướng thiết kế gần với Transformer. Mô hình có khả năng học đặc trưng ảnh tự nhiên tốt, đồng thời vẫn giữ được cấu trúc tương đối gọn để tích hợp vào ứng dụng demo.

So với ResNet18, ConvNeXt-Tiny phù hợp hơn với bài toán severity classification của dự án vì có khả năng biểu diễn đặc trưng mạnh hơn, đặc biệt với các chi tiết hư hỏng nhỏ, vùng mờ hoặc xước mờ và các trường hợp nhập nhằng giữa minor, moderate và severe. Trong dự án, ResNet18 chỉ được xem là mô hình thử nghiệm/baseline ban đầu, không phải mô hình triển khai chính.

Kết quả severity từ ConvNeXt-Tiny được đưa vào tầng điều chỉnh giá cùng với base price từ XGBoost và damage features từ YOLOv8s. Nhờ đó, hệ thống không chỉ biết xe có những hư hỏng nào và diện tích tương đối ra sao, mà còn có thêm tín hiệu về mức độ nghiêm trọng tổng quát để tính giá tham khảo sau điều chỉnh.

2.5. Các metric: RMSE, MAE, R^2 , mAP, accuracy

Trong dự án, hệ thống gồm nhiều mô hình đảm nhiệm các nhiệm vụ khác nhau, bao gồm mô hình hồi quy để dự đoán giá xe, mô hình phát hiện hư hỏng bằng YOLO và mô hình phân loại mức độ hư hỏng bằng CNN/ConvNeXt-Tiny. Vì mỗi bài toán có bản chất khác nhau nên cần sử dụng các nhóm chỉ số đánh giá khác nhau để phản ánh đúng hiệu quả của từng thành phần.

2.5.1. Metrics Cho Bài Toán Dự Đoán Giá Xe

Đối với mô hình dự đoán giá xe từ dữ liệu dạng bảng, đây là bài toán hồi quy nên dự án sử dụng các chỉ số như MAE, RMSE và R^2 Score.

- MAE, viết tắt của Mean Absolute Error, là sai số tuyệt đối trung bình giữa giá trị dự đoán và giá trị thực tế. Chỉ số này cho biết trung bình mô hình dự đoán lệch bao nhiêu so với giá trị thật. MAE càng nhỏ thì mô hình càng dự đoán gần với thực tế. Trong bài toán định giá xe cũ, MAE giúp đánh giá mức sai lệch trung bình của mô hình theo đơn vị giá.
- RMSE, viết tắt của Root Mean Squared Error, là căn bậc hai của trung bình bình phương sai số. So với MAE, RMSE nhạy hơn với các dự đoán sai lệch lớn vì sai số được bình phương trước khi lấy trung bình. Do đó, RMSE phù hợp để kiểm tra xem mô hình có tạo ra những dự đoán lệch quá xa so với giá trị thật hay không. RMSE càng nhỏ thì mô hình càng ổn định.
- R^2 Score là hệ số xác định, dùng để đo mức độ mô hình giải thích được sự biến thiên của dữ liệu mục tiêu. Giá trị R^2 càng gần 1 thì mô hình càng phù hợp với dữ liệu. Trong dự án, R^2 được dùng để so sánh khả năng học quan hệ giữa các đặc trưng như năm sản xuất, số km đã đi, loại nhiên liệu, hộp số, hãng xe và giá xe.

2.5.2 Metrics Cho Bài Toán Object Detection Bằng YOLO

Đối với mô hình YOLOv8, nhiệm vụ của mô hình không chỉ là phân loại đúng loại hư hỏng mà còn phải xác định đúng vị trí vùng hư hỏng trên ảnh. Vì vậy, các chỉ số đánh giá thường dùng gồm Precision, Recall, $mAP@0.5$ và [\$mAP@0.5:0.95\$](#) .

Precision cho biết trong số các vùng hư hỏng mà mô hình dự đoán, có bao nhiêu vùng là đúng. Precision cao nghĩa là mô hình ít báo nhầm. Trong bài toán phát hiện hư hỏng xe, điều này giúp giảm tình trạng nhận nhầm các chi tiết bình thường như bóng phản sáng, đường viền thân xe hoặc vết bẩn thành hư hỏng.

Recall cho biết trong số các vùng hư hỏng thực sự tồn tại trên ảnh, mô hình phát hiện được bao nhiêu vùng. Recall cao nghĩa là mô hình ít bỏ sót hư hỏng. Đây là chỉ số quan trọng vì nếu hệ thống bỏ sót nhiều vùng hỏng thì bước điều chỉnh giá sau đó sẽ thiếu thông tin.

$mAP@0.5$, hay mean Average Precision tại ngưỡng IoU 0.5, là chỉ số tổng hợp giữa khả năng phân loại đúng và định vị đúng bounding box. Một dự đoán được xem là đúng khi bounding box dự đoán có độ giao nhau đủ lớn với bounding box thật. Trong dự án, $mAP@0.5$ được dùng làm chỉ số chính để đánh giá chất lượng phát hiện hư hỏng của YOLO.

mAP@0.5:0.95 là phiên bản đánh giá khắt khe hơn, tính trung bình mAP trên nhiều ngưỡng IoU từ 0.5 đến 0.95. Chỉ số này phản ánh tốt hơn độ chính xác của vị trí bounding box. Nếu mAP@0.5:0.95 cao, điều đó cho thấy mô hình không chỉ phát hiện đúng vùng hồng mà còn khoanh vùng tương đối chính xác.

2.5.3. Metrics cho bài toán phân loại mức độ hư hỏng

Đối với mô hình CNN/ConvNeXt-Tiny, nhiệm vụ là phân loại ảnh full do người dùng upload thành các mức minor, moderate và severe. Đây là bài toán phân loại nhiều lớp, vì vậy có thể sử dụng các chỉ số như Accuracy, Precision, Recall, F1-score và Confusion Matrix.

Accuracy là tỷ lệ dự đoán đúng trên tổng số mẫu. Đây là chỉ số dễ hiểu và phù hợp để đánh giá tổng quan mô hình phân loại. Tuy nhiên, nếu dữ liệu giữa các lớp không cân bằng, Accuracy có thể chưa phản ánh đầy đủ chất lượng mô hình.

Precision trong bài toán phân loại cho biết trong số các mẫu được dự đoán thuộc một mức độ hư hỏng, có bao nhiêu mẫu thực sự thuộc mức đó. Chỉ số này giúp đánh giá mức độ đáng tin cậy của từng nhãn mà mô hình đưa ra.

Recall cho biết trong số các mẫu thực sự thuộc một mức độ hư hỏng, mô hình nhận diện đúng được bao nhiêu mẫu. Với dự án này, Recall đặc biệt quan trọng ở lớp severe, vì bỏ sót các hư hỏng nghiêm trọng có thể khiến hệ thống đánh giá mức giảm giá thấp hơn thực tế.

F1-score là trung bình điều hòa giữa Precision và Recall. Chỉ số này hữu ích khi cần đánh giá cân bằng giữa việc hạn chế dự đoán sai và hạn chế bỏ sót. Với bài toán phân loại mức độ hư hỏng, F1-score giúp đánh giá mô hình tốt hơn so với chỉ dùng Accuracy.

Confusion Matrix là ma trận thể hiện số lượng mẫu của từng lớp được dự đoán đúng hoặc nhầm sang lớp khác. Ma trận này giúp quan sát cụ thể mô hình thường nhầm giữa những mức nào, ví dụ nhầm minor với moderate hoặc moderate với severe. Đây là thông tin quan trọng để cải thiện dữ liệu và mô hình trong các phiên bản sau.

2.5.4. Vai trò của metrics trong dự án

Việc sử dụng nhiều nhóm metrics giúp đánh giá hệ thống một cách toàn diện hơn. Mô hình dự đoán giá được đánh giá bằng các chỉ số hồi quy, YOLO được đánh giá bằng các chỉ số phát hiện đối tượng, còn CNN được đánh giá bằng các chỉ số phân loại ảnh. Nhờ đó, dự án có thể xác định rõ thành phần nào hoạt động tốt, thành phần nào còn hạn chế và cần cải thiện.

Trong hệ thống hiện tại, kết quả cuối cùng phụ thuộc vào cả ba nhánh: dự đoán giá cơ bản, phát hiện hư hỏng và phân loại mức độ hư hỏng. Nếu một trong các mô hình hoạt động chưa tốt, kết quả định giá tham khảo cuối cùng cũng có thể bị ảnh hưởng. Vì vậy, việc theo dõi các metrics riêng cho từng mô hình là cần thiết để đảm bảo pipeline hoạt động ổn định và có cơ sở đánh giá rõ ràng.

3. Dataset và tiền xử lý

Trong dự án, dữ liệu được tổ chức theo hướng mô-đun, tương ứng với các nhiệm vụ khác nhau trong pipeline định giá xe ô tô cũ. Dự án kết hợp ba nhóm dữ liệu chính: dữ liệu bảng phục vụ dự đoán giá xe cơ bản, dữ liệu ảnh có annotation phục vụ phát hiện hư hỏng ngoại thất bằng YOLOv8s, và dữ liệu ảnh full được gán nhãn severity phục vụ phân loại mức độ hư hỏng bằng CNN/ConvNeXt-Tiny.

Cách tổ chức này phù hợp với mục tiêu của dự án vì bài toán định giá xe cũ có xét đến hư hỏng là một bài toán gồm nhiều thành phần. Dữ liệu bảng giúp mô hình học mối quan hệ giữa các thông tin kỹ thuật của xe và giá thị trường. Trong khi đó, dữ liệu ảnh giúp hệ thống khai thác thêm tình trạng ngoại thất thực tế của xe, bao gồm vị trí hư hỏng, loại hư hỏng và mức độ nghiêm trọng.

3.1. Dataset giá xe cũ dạng bảng

Dataset giá xe cũ được lưu dưới dạng file CSV, bao gồm các thông tin mô tả xe như tên xe, năm sản xuất, số km đã đi, loại nhiên liệu, hộp số, số chủ sở hữu, số chỗ ngồi, mức tiêu hao nhiên liệu, dung tích động cơ, công suất và giá xe. Trong đó, cột giá xe được sử dụng làm biến mục tiêu cho bài toán hồi quy.

Nguồn dữ liệu bảng được sử dụng trong dự án là bộ Used Cars Price Prediction trên Kaggle: <https://www.kaggle.com/datasets/avikasliwal/used-cars-price-prediction>. Bộ dữ liệu này phù hợp với nhánh tabular price prediction vì chứa các thuộc tính thường ảnh hưởng trực tiếp đến giá xe cũ, như năm sản xuất, số km đã đi, nhiên liệu, hộp số, động cơ, công suất và giá bán.

Mục đích của dataset này là huấn luyện mô hình dự đoán base price, tức là mức giá cơ bản của xe trước khi xét đến tình trạng hư hỏng ngoại thất. Đây là đầu vào quan trọng cho bước tính giá tham khảo sau điều chỉnh của hệ thống.

3.1.1. Quy mô và cấu trúc tổng quát của dataset

Bộ dữ liệu giá xe cũ được cung cấp dưới dạng hai file riêng biệt gồm tập huấn luyện và tập kiểm thử. Hai tập dữ liệu có cấu trúc thuộc tính gần giống nhau,

nhưng khác nhau ở việc tập huấn luyện có cột giá xe, trong khi tập kiểm thử không chứa cột này.

- **Tập train** gồm **6.019 dòng và 14 cột**. Mỗi dòng đại diện cho một xe ô tô cũ, bao gồm các thông tin về tên xe, địa điểm đăng bán, năm sản xuất, số kilomet đã đi, loại nhiên liệu, hộp số, số đời chủ sở hữu, mức tiêu hao nhiên liệu, dung tích động cơ, công suất, số chỗ ngồi, giá xe mới và giá xe cũ. Trong đó, cột Price là biến mục tiêu được sử dụng để huấn luyện và đánh giá các mô hình hồi quy.
- **Tập test** gồm **1.234 dòng và 13 cột**. Tập này chứa các thuộc tính mô tả xe tương tự tập train nhưng không có cột Price. Do không có giá trị mục tiêu, tập test được sử dụng để đưa vào mô hình đã huấn luyện và sinh ra kết quả dự đoán giá, thay vì sử dụng để tính trực tiếp các chỉ số đánh giá như MAE, RMSE hoặc R^2 .

Sự khác biệt về số lượng cột giữa hai tập dữ liệu được thể hiện như sau:

Tập dữ liệu	Số dòng	Số cột	Có cột Price	Mục đích sử dụng
Train	6019	14	Có	Huấn luyện mô hình, chia tập validation và đánh giá mô hình
Test	1234	13	Không	Sinh dự đoán giá cho các mẫu chưa biết kết quả

```

train = pd.read_csv("/kaggle/input/used-cars-price-prediction/train-data.csv")
test = pd.read_csv("/kaggle/input/used-cars-price-prediction/test-data.csv")
print(train.count())
test.count()

```

```

Unnamed: 0      6019
Name            6019
Location        6019
Year            6019
Kilometers_Driven 6019
Fuel_Type       6019
Transmission     6019
Owner_Type       6019
Mileage          6017
Engine           5983
Power            5983
Seats           5977
New_Price        824
Price            6019
dtype: int64
[5]: Unnamed: 0      1234
Name            1234
Location        1234
Year            1234
Kilometers_Driven 1234
Fuel_Type       1234
Transmission     1234
Owner_Type       1234
Mileage          1234
Engine           1224
Power            1224
Seats           1223
New_Price        182
dtype: int64

```

Hình 7: Thống kê số lượng bản ghi hợp lệ theo từng cột của tập train và tập test

Trong các cột ban đầu có một cột chỉ mục là Unnamed: 0, được sinh ra trong quá trình lưu dữ liệu dưới dạng CSV. Cột này chỉ biểu diễn số thứ tự của dòng, không chứa thông tin liên quan đến đặc điểm hoặc giá trị của xe nên được loại bỏ trước khi huấn luyện mô hình.

3.1.2. Ý nghĩa của các cột dữ liệu

Cấu trúc chi tiết của dataset được trình bày trong bảng:

Tên cột	Kiểu dữ liệu ban đầu	Ý nghĩa	Vai trò trong dự án
Unnamed: 0	Số nguyên	Số thứ tự của dòng khi file CSV được lưu	Không mang ý nghĩa dự báo, được loại bỏ

Tên cột	Kiểu dữ liệu ban đầu	Ý nghĩa	Vai trò trong dự án
Name	Chuỗi	Tên đầy đủ của xe, thường gồm hãng, dòng xe và phiên bản	Được tách để lấy hãng xe và nhóm tên xe
Location	Phân loại	Thành phố hoặc khu vực nơi xe được đăng bán	Phản ánh sự khác biệt thị trường giữa các khu vực
Year	Số nguyên	Năm sản xuất của xe	Dùng để tạo đặc trưng tuổi xe
Kilometers_Driven	Số nguyên	Tổng số kilomet xe đã vận hành	Phản ánh mức độ sử dụng và hao mòn của xe
Fuel_Type	Phân loại	Loại nhiên liệu, ví dụ Petrol, Diesel, CNG, LPG hoặc Electric	Được mã hóa thành dữ liệu số
Transmission	Phân loại	Loại hộp số, chủ yếu gồm Manual và Automatic	Được mã hóa thành dữ liệu số
Owner_Type	Phân loại thứ bậc	Số đời chủ sở hữu, chẳng hạn First, Second, Third hoặc Fourth & Above	Phản ánh lịch sử sở hữu của xe
Mileage	Chuỗi chứa số và đơn vị	Mức tiêu hao hoặc hiệu suất nhiên liệu, thường có đơn vị kmpl hoặc km/kg	Được tách phần số và chuyển sang dạng số
Engine	Chuỗi chứa số và đơn vị	Dung tích động cơ, thường được biểu diễn bằng CC	Được tách phần số và chuyển sang dạng số

Tên cột	Kiểu dữ liệu ban đầu	Ý nghĩa	Vai trò trong dự án
Power	Chuỗi chứa số và đơn vị	Công suất động cơ, thường được biểu diễn bằng bhp	Được tách phần số và chuyển sang dạng số
Seats	Số thực	Số chỗ ngồi của xe	Dùng trực tiếp sau khi xử lý giá trị thiếu
New_Price	Chuỗi chứa số và đơn vị tiền tệ	Giá tham khảo của xe khi còn mới	Có nhiều giá trị thiếu, cần kiểm tra trước khi sử dụng
Price	Số thực	Giá xe cũ được ghi nhận trong bộ dữ liệu, theo đơn vị lakh	Biến mục tiêu của bài toán hồi quy

Các cột trong dataset có thể được chia thành bốn nhóm chính:

- **Thông tin nhận dạng:** Name, Location.
- **Thông tin lịch sử sử dụng:** Year, Kilometers_Driven, Owner_Type.
- **Thông số kỹ thuật:** Fuel_Type, Transmission, Mileage, Engine, Power, Seats.
- **Thông tin giá:** New_Price, Price.

Trong đó, Price là biến phụ thuộc cần dự đoán, còn các cột còn lại là các biến độc lập có thể được sử dụng để xây dựng đặc trưng đầu vào.

3.2. Dataset phát hiện hư hỏng xe

3.2.1. Nguồn dữ liệu

Dataset phát hiện hư hỏng được sử dụng để huấn luyện mô hình YOLOv8 cho bài toán object detection. Dữ liệu bao gồm ảnh xe và annotation mô tả vị trí các vùng hư hỏng trên ảnh. Mỗi vùng hư hỏng được biểu diễn bằng bounding box và nhãn class tương ứng.

Nguồn dữ liệu ban đầu cho nhánh phát hiện hư hỏng là bộ Car Damage Detection trên Kaggle:

<https://www.kaggle.com/asfarhossainsitab/car-damage-detection>.

Khi tải về, annotation của dataset gốc ở định dạng COCO, trong khi YOLOv8 yêu cầu nhãn theo định dạng YOLO. Vì vậy, em đã đưa dữ liệu lên Roboflow để kiểm tra annotation và export lại theo định dạng YOLOv8 trước khi huấn luyện bằng Ultralytics.

Các lớp hư hỏng trong dataset bao gồm những dạng hư hỏng thường gặp trên xe như vết móp, vết xước, vết nứt, kính vỡ hoặc đèn hỏng. Đây là các loại hư hỏng có đặc điểm thị giác khác nhau, trong đó một số lớp như scratch, crack, dent thường khó phát hiện hơn vì vùng hỏng nhỏ, mảnh hoặc dễ bị nhầm với phản sáng và đường nét thân xe.

Ở giai đoạn sau, để tăng độ đa dạng dữ liệu và cải thiện khả năng nhận diện các lớp khó, dự án bổ sung thêm 2307 ảnh từ Roboflow Universe. Hai nguồn bổ sung gồm:

AutoDentify - Car Damage Detection:

<https://universe.roboflow.com/autodentify/car-damage-detection-ggmju>

Scratch and Dent: <https://universe.roboflow.com/nibm-7v215/scratch-and-dent-xvjy5>

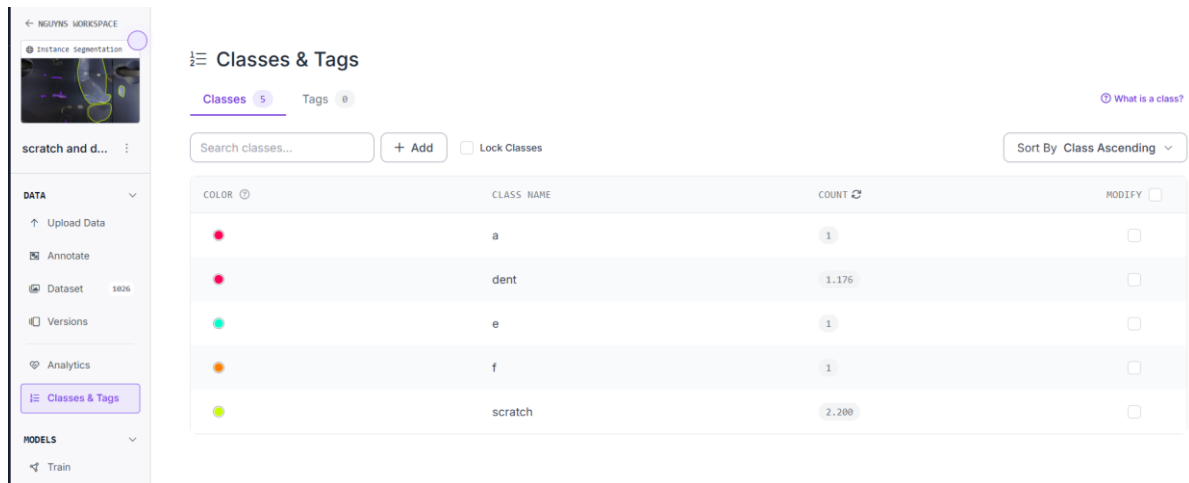
Phần dữ liệu bổ sung tập trung nhiều vào các hư hỏng như dent, scratch và crack, là những lớp có vùng hỏng nhỏ, ranh giới mờ và dễ bị bỏ sót trong ảnh thực tế. Sau khi ghép dataset gốc với dữ liệu bổ sung từ Roboflow Universe, dự án tạo ra phiên bản dữ liệu merge_data để huấn luyện YOLOv8s. Phiên bản này giúp mô hình có thêm ví dụ học cho các tình huống hư hỏng đa dạng hơn, từ đó phù hợp hơn với mục tiêu tích hợp vào hệ thống demo định giá xe cũ.

Dữ liệu ban đầu được chuyển đổi sang định dạng YOLO để tương thích với quá trình huấn luyện bằng thư viện Ultralytics. Trong định dạng này, mỗi ảnh sẽ có một file nhãn tương ứng, chứa thông tin class và tọa độ bounding box đã được chuẩn hóa. Dataset được chia thành các tập train, val và test, trong đó tập train dùng để học tham số, tập validation dùng để theo dõi chất lượng mô hình trong quá trình huấn luyện, còn tập test dùng để đánh giá cuối cùng.

Trong quá trình tiền xử lý, ảnh và nhãn được kiểm tra để đảm bảo annotation khớp với ảnh, class hợp lệ và không thiếu file nhãn quan trọng. Ảnh được resize theo kích thước đầu vào của mô hình, cụ thể trong dự án là cấu hình yolov8_merge_data_1061, sử dụng YOLOv8s, huấn luyện 100 epoch với kích thước ảnh 640px. Kích thước này giúp giữ lại nhiều chi tiết hơn cho các vùng hư hỏng nhỏ.

3.2.2. Phân bố nhãn hư hỏng

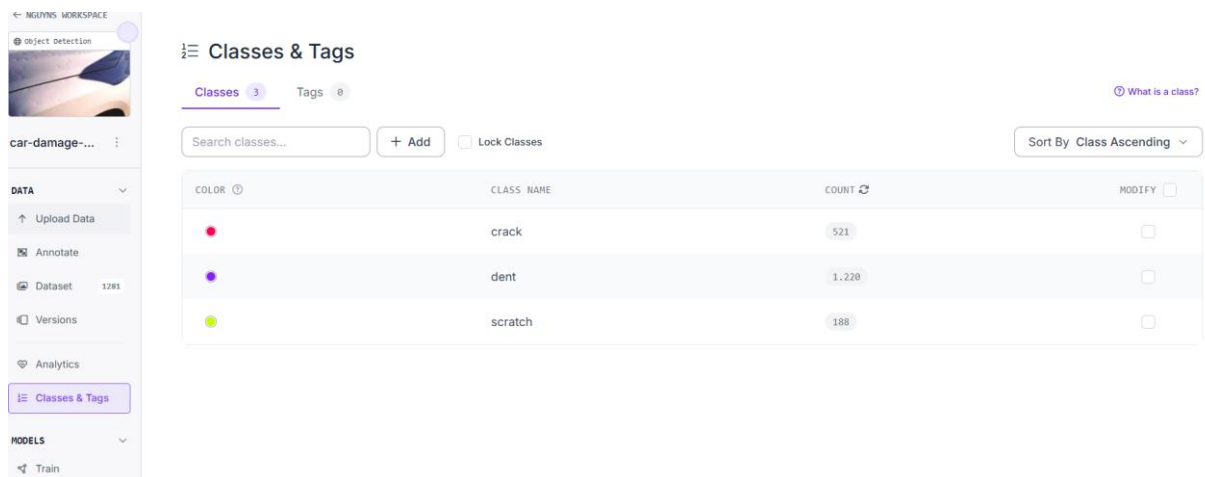
Số bounding box của dataset 1026 ảnh



COLOR	CLASS NAME	COUNT	MODIFY
	a	1	☐
	dent	1.176	☐
	e	1	☐
	f	1	☐
	scratch	2.200	☐

Hình 8: Phân bố số lượng bounding box theo lớp của dataset 1.026 ảnh

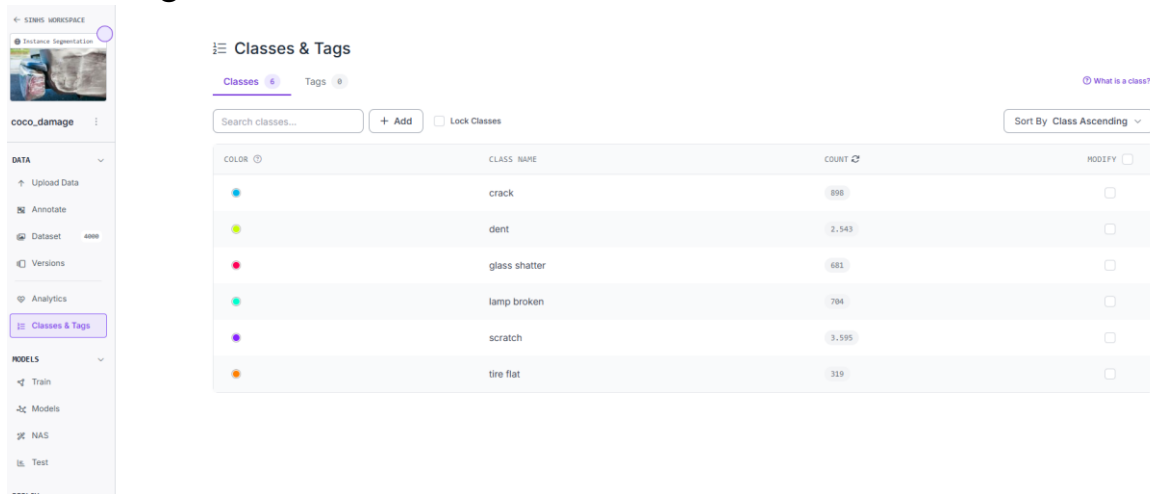
Số bounding box của dataset 1281 ảnh



COLOR	CLASS NAME	COUNT	MODIFY
	crack	521	☐
	dent	1.228	☐
	scratch	188	☐

Hình 9: Phân bố số lượng bounding box theo lớp của dataset 1.281 ảnh

Số bounding box của dataset 4000 ảnh



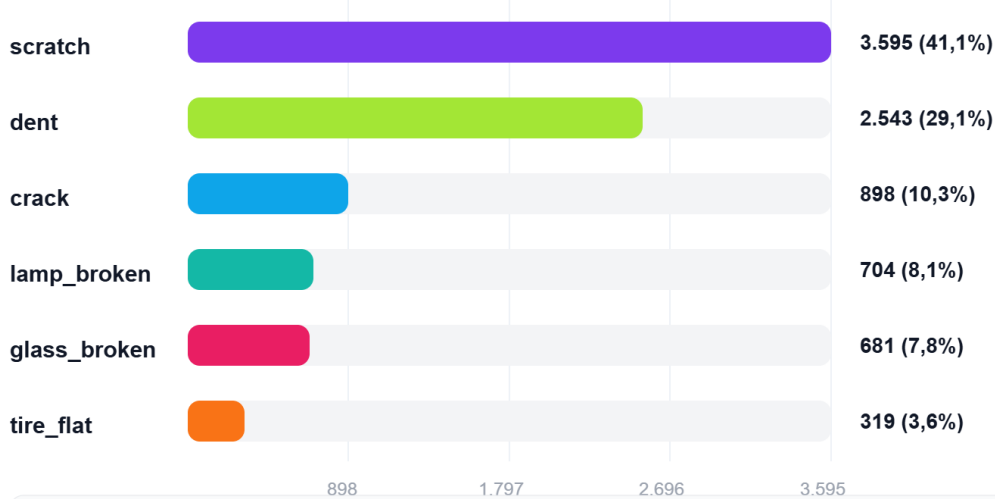
Hình 10: Phân bố số lượng bounding box theo lớp của dataset khoảng 4.000 ảnh

Từ việc phân bố nhãn như trên, ta có bảng tổng hợp số lượng bounding box cho toàn bộ datasets:

Phân bố nhãn YOLO trong dataset ảnh

merge_Data - bỏ các lớp nhiễu a, e, f từ nguồn Roboflow phụ

Tổng số annotation: 8.740

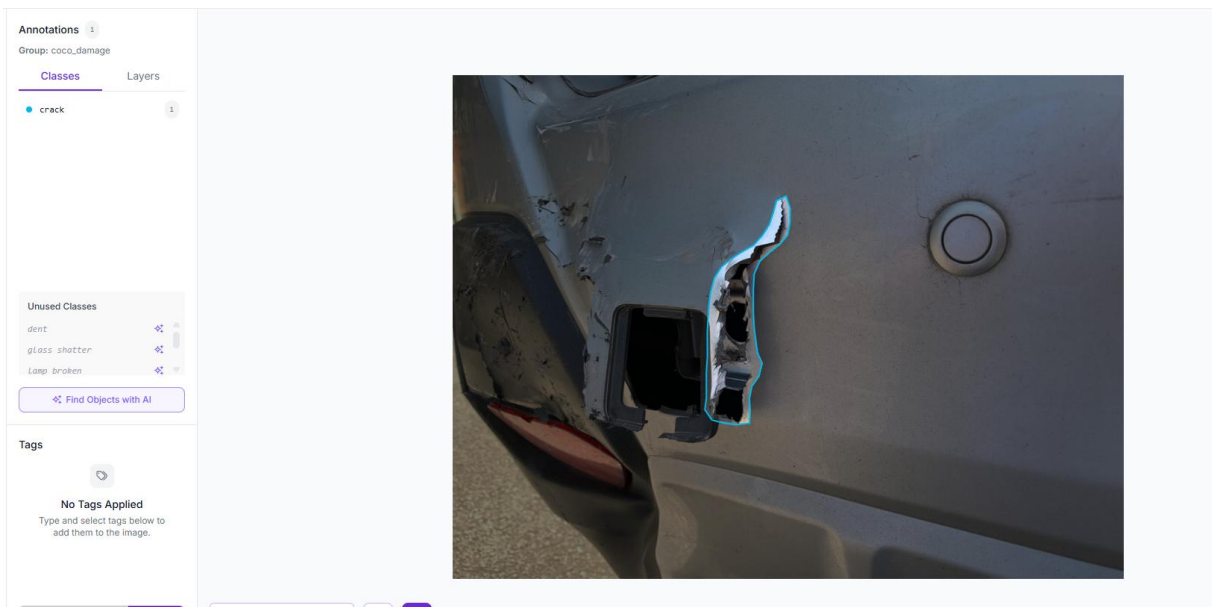


Ghi chú: số lượng thể hiện số annotation/nhãn theo lớp trong Roboflow, không nhất thiết bằng số ảnh.

Hình 11: Phân bố tổng hợp số lượng bounding box theo lớp trong dataset merge_data

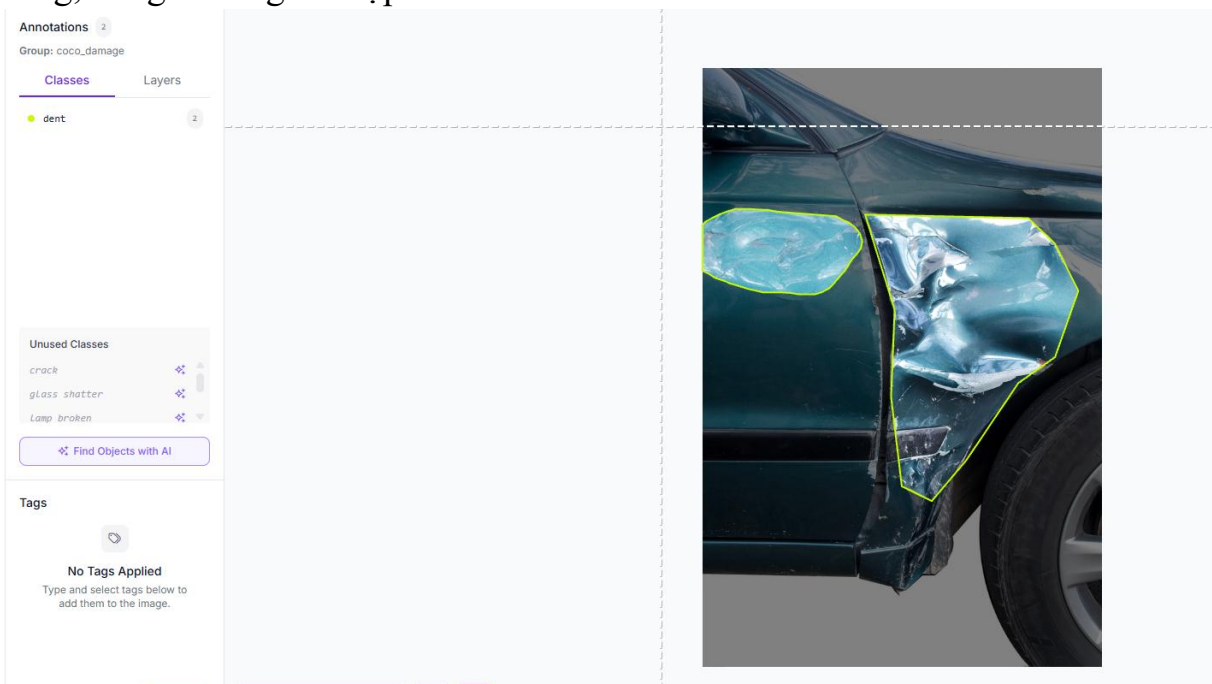
3.2.3. Đặc điểm thị giác của từng lớp hư hỏng

Crack – vết nứt: thường xuất hiện dưới dạng đường gãy, đường rạn hoặc vùng vật liệu bị tách. Vết nứt có thể nhỏ, mảnh và dễ bị nhầm với đường phản sáng, khe nổi hoặc hoa văn trên thân xe.



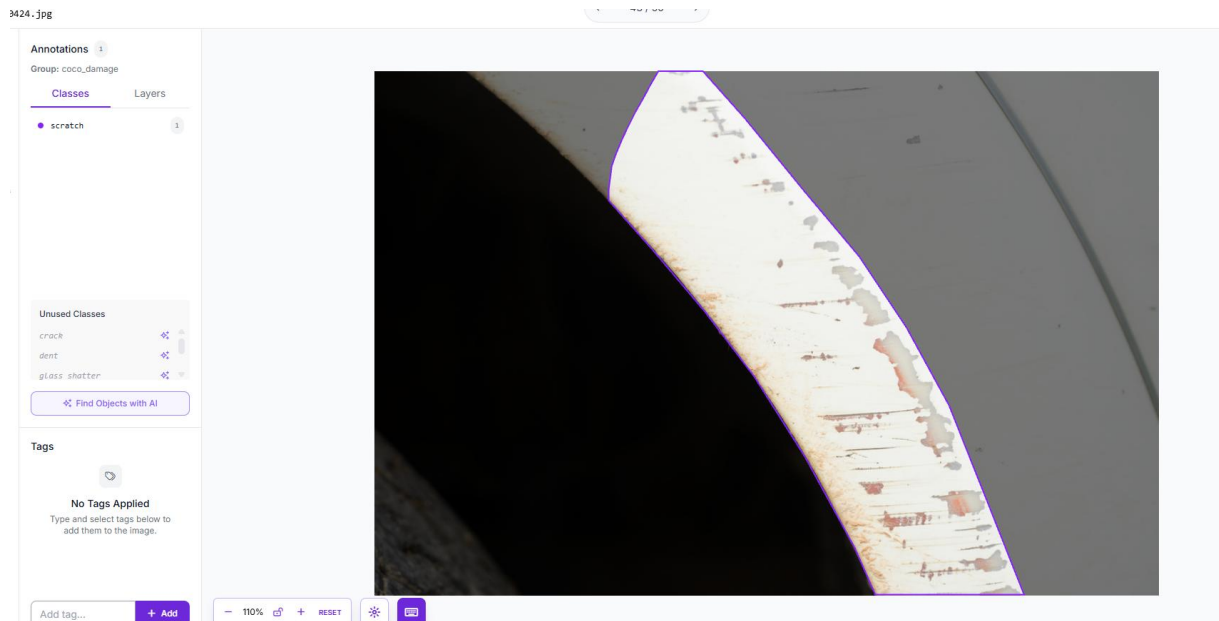
Hình 12: Ảnh minh họa lớp crack (vết nứt)

Dent – vết móp: là vùng bề mặt thân xe bị biến dạng, lõm vào hoặc cong bất thường. Dent thường không có đường biên rõ ràng và phụ thuộc nhiều vào ánh sáng, bóng đổ và góc chụp.



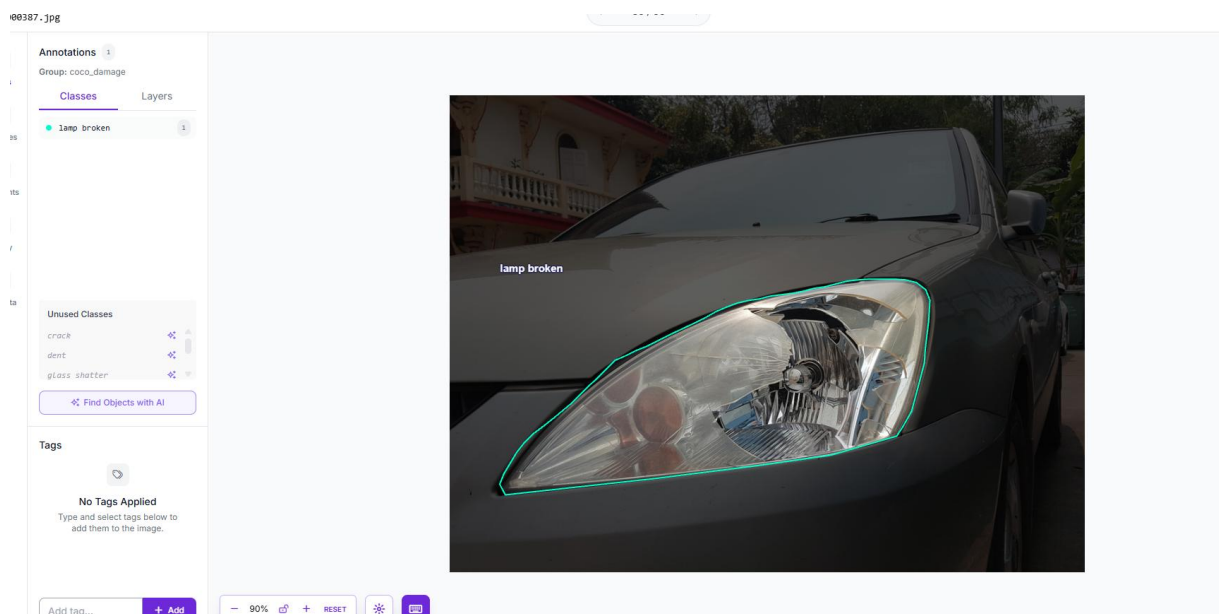
Hình 13: Ảnh minh họa lớp dent (vết móp)

Scratch – vết xước: thường là các đường dài, mảnh hoặc vùng sơn bị tróc. Scratch nhỏ dễ bị nhầm với bụi bẩn, ánh sáng phản chiếu hoặc đường trang trí trên xe.



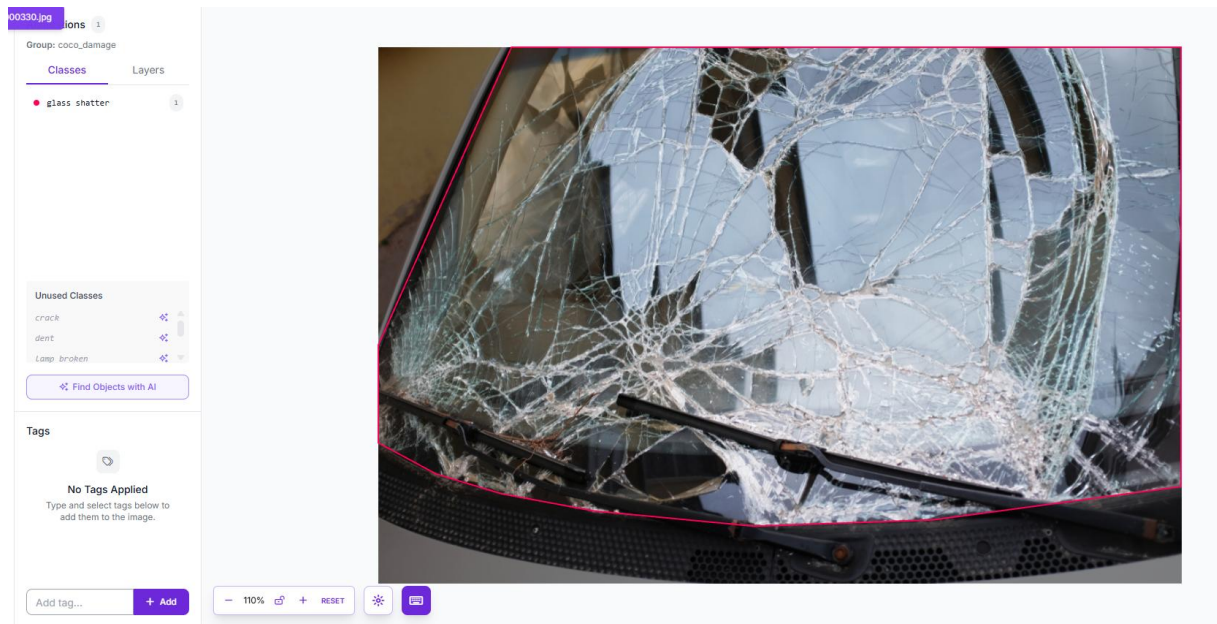
Hình 14: Ảnh minh họa lớp scratch (vết xước)

Lamp broken – đèn bị hỏng: thể hiện qua đèn nứt, vỡ, mất một phần vỏ hoặc cụm đèn bị biến dạng.



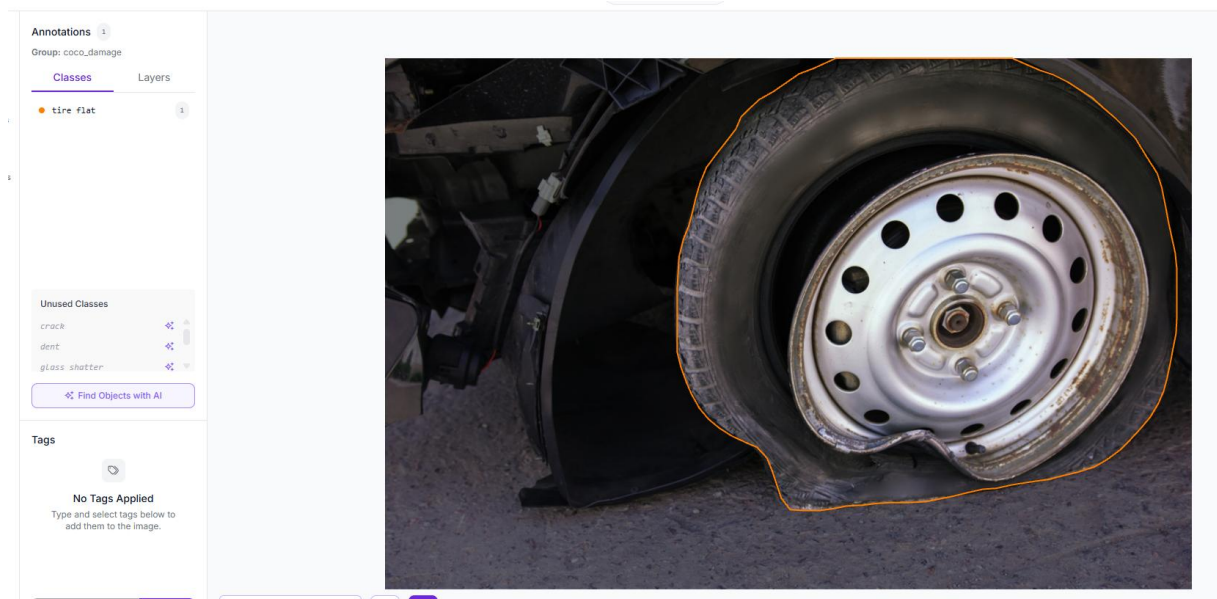
Hình 15: Ảnh minh họa lớp lamp_broken (đèn bị hỏng)

Glass shatter– kính bị vỡ: gồm kính nứt, kính vỡ hoặc xuất hiện mạng lưới đường rạn trên bề mặt kính.



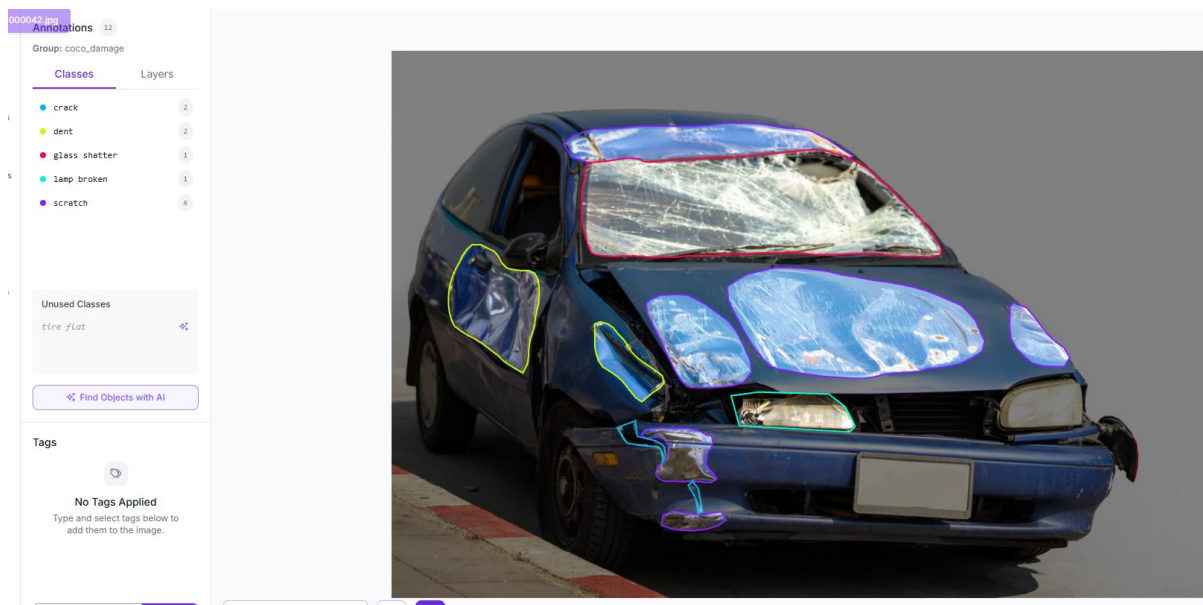
Hình 16: Ảnh minh họa lớp `glass_shatter` (kính bị vỡ)

Tire flat – lốp xẹp: lốp bị mất áp suất, biến dạng hoặc phần thân xe thấp bất thường so với mặt đường.



Hình 17: Ảnh minh họa lớp `tire_flat` (lốp bị xẹp)

Trên cùng 1 xe có thể có rất nhiều loại hư hỏng khác nhau



Hình 18: Ảnh xe có đồng thời nhiều loại hư hỏng ngoại thất

3.2.4. Độ đa dạng về góc chụp và điều kiện ảnh

Bộ dữ liệu merge_data được tổng hợp từ ba nhóm dữ liệu ảnh có đặc điểm không hoàn toàn giống nhau. Cụ thể, dự án sử dụng một bộ dữ liệu ban đầu gồm khoảng 4.000 ảnh, một bộ dữ liệu bổ sung gồm 1.026 ảnh và một bộ dữ liệu bổ sung khác gồm 1.281 ảnh. Sự khác biệt giữa các nguồn không chỉ nằm ở số lượng ảnh mà còn thể hiện ở kiểu annotation, chất lượng hình ảnh, khoảng cách chụp và tỷ lệ vùng hư hỏng so với toàn bộ ảnh.

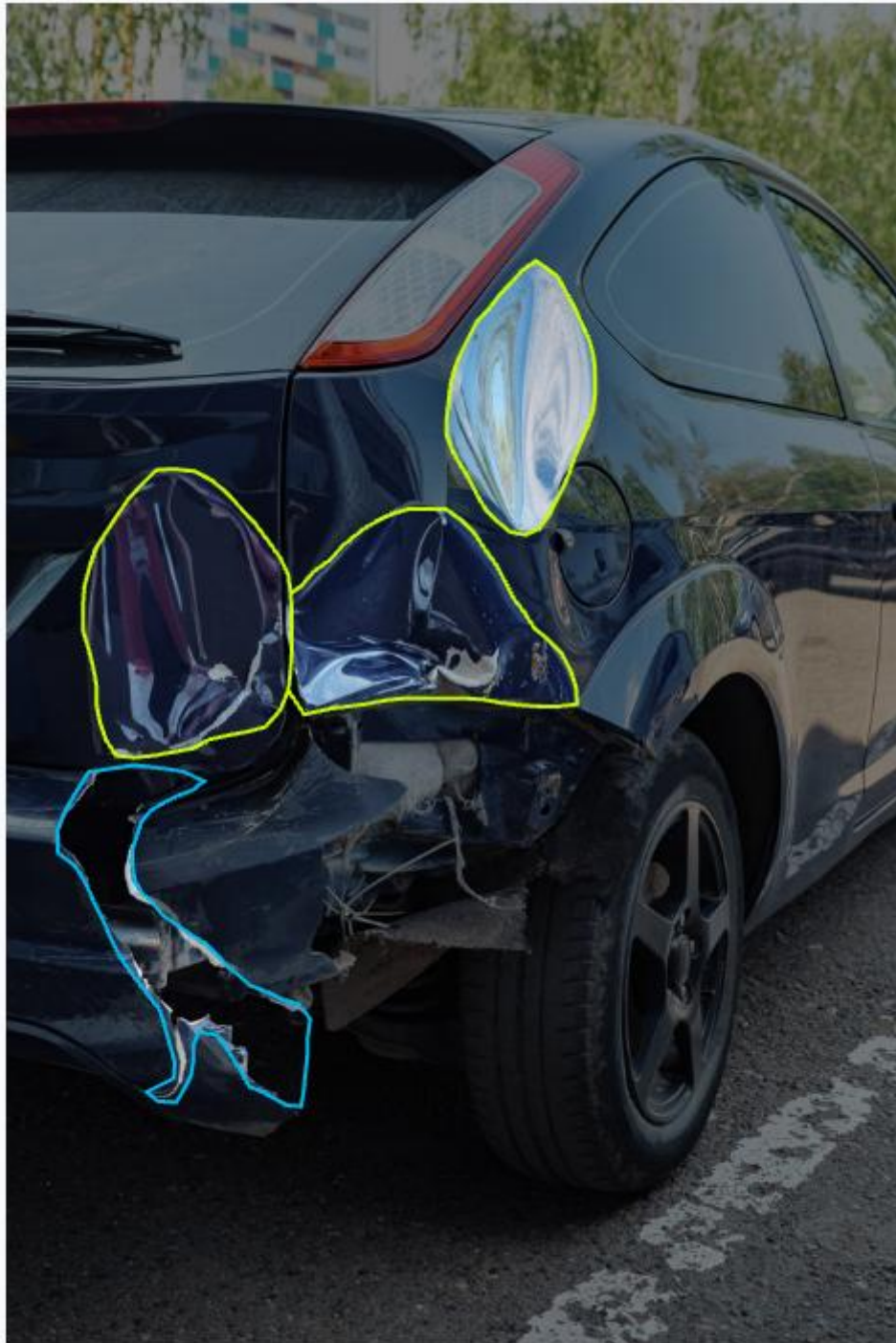
Bảng dưới đây tóm tắt đặc điểm chính của các nguồn dữ liệu:

Nhóm dữ liệu	Số lượng ảnh	Kiểu annotation ban đầu	Đặc điểm chất lượng ảnh	Xu hướng góc chụp
Dataset ban đầu	Khoảng 4.000	Instance segmentation	Ảnh tương đối rõ nét, vùng hư hỏng thể hiện chi tiết	Chủ yếu chụp gần hoặc cận cảnh vùng hư hỏng
Dataset bổ sung thứ nhất	1.026	Instance segmentation	Chất lượng ảnh khá tốt, đường biên vùng hỏng tương đối rõ	Có xu hướng chụp gần, vùng hỏng chiếm tỷ lệ lớn trong ảnh

Dataset bổ sung thứ hai	1.281	Object detection	Chất lượng ảnh thấp hơn, một số ảnh có độ phân giải thấp hoặc bị mờ	Có xu hướng chụp xa, thể hiện toàn cảnh hoặc phần lớn chiếc xe
-------------------------	-------	------------------	---	--

Hai bộ dữ liệu gồm khoảng 4.000 ảnh và 1.026 ảnh sử dụng annotation dạng **instance segmentation**. Với dạng annotation này, vùng hư hỏng được đánh dấu theo đường biên tương đối sát với hình dạng thực tế, thay vì chỉ được bao quanh bằng bounding box hình chữ nhật. Điều này giúp thể hiện chính xác hơn phạm vi của các hư hỏng có hình dạng không đều như vùng móp, phần thân xe bị biến dạng hoặc khu vực bị vỡ.

Qua quan sát định tính, ảnh trong hai bộ dữ liệu instance segmentation có chất lượng tương đối tốt. Nhiều ảnh có độ nét cao, ánh sáng đủ và vùng hư hỏng được chụp ở khoảng cách gần. Do vùng hỏng chiếm tỷ lệ tương đối lớn trong khung hình nên các đặc điểm như sự biến dạng của bề mặt, vết nứt, vết xước hoặc phần vật liệu bị vỡ được thể hiện rõ hơn. Đây là điều kiện thuận lợi để mô hình học các đặc trưng cục bộ của hư hỏng.



Hình 19: Ảnh từ nhóm dữ liệu instance segmentation, được chụp gần và có độ nét cao

Trong ảnh minh họa, chiếc xe được chụp ở góc chéo phía sau và phần hư hỏng chiếm diện tích lớn trong ảnh. Các khu vực thân xe bị móp, biến dạng và phần cản sau bị vỡ được thể hiện tương đối rõ. Annotation dạng segmentation bám theo hình dạng của từng vùng hỏng, giúp mô tả chi tiết hơn so với bounding box thông thường.

Tuy nhiên, việc phần lớn ảnh được chụp gần cũng có thể tạo ra một hạn chế. Mô hình có thể học tốt các hư hỏng khi chúng xuất hiện rõ và chiếm tỷ lệ lớn trong ảnh, nhưng có thể gặp khó khăn khi người dùng tải lên ảnh chụp toàn bộ xe, trong đó vùng hư hỏng chỉ chiếm một phần nhỏ. Vì vậy, dữ liệu chụp cận cảnh có lợi cho việc học đặc trưng chi tiết nhưng chưa phản ánh đầy đủ mọi tình huống sử dụng thực tế.

Ngược lại, bộ dữ liệu gồm 1.281 ảnh sử dụng annotation dạng **object detection**, trong đó mỗi vùng hư hỏng được bao quanh bởi một bounding box. So với instance segmentation, bounding box đơn giản hơn nhưng không mô tả chính xác đường biên của vùng hư hỏng. Đối với các hư hỏng có hình dạng phức tạp hoặc kéo dài, bounding box có thể bao gồm cả một phần nền hoặc các bộ phận không bị hỏng.

Chất lượng hình ảnh của bộ 1.281 ảnh nhìn chung thấp hơn so với hai bộ instance segmentation. Một số ảnh có độ phân giải thấp, bị nén hoặc không thể hiện rõ chi tiết bề mặt xe. Các ảnh cũng có xu hướng được chụp ở khoảng cách xa hơn, thường thể hiện toàn bộ xe hoặc một phần lớn thân xe. Do đó, vùng hư hỏng có thể chiếm diện tích nhỏ trong ảnh và khó quan sát bằng mắt thường.



Hình 20: Ảnh từ nhóm dữ liệu object detection, được chụp xa và có chất lượng thấp hơn

Trong ảnh minh họa, xe được chụp theo góc bên hông và gần như toàn bộ phần thân bên của xe xuất hiện trong khung hình. Vùng được annotation chỉ chiếm một phần nhỏ so với kích thước toàn ảnh. Độ nét của ảnh thấp hơn và có nhiều

chi tiết nền xung quanh, khiến việc xác định ranh giới chính xác của vùng hư hỏng trở nên khó khăn hơn.

Mặc dù có chất lượng thấp hơn, bộ dữ liệu object detection vẫn có vai trò quan trọng đối với quá trình huấn luyện. Các ảnh chụp xa giúp mô hình làm quen với tình huống vùng hư hỏng xuất hiện ở kích thước nhỏ trong ảnh toàn cảnh. Đây là trường hợp có thể xảy ra trong ứng dụng thực tế khi người dùng chụp toàn bộ xe thay vì chụp riêng từng vùng hỏng. Vì vậy, bộ dữ liệu 1.281 ảnh góp phần bổ sung sự đa dạng về khoảng cách chụp, tỷ lệ đối tượng và bối cảnh ảnh.

Sự kết hợp giữa các nguồn dữ liệu tạo ra sự đa dạng về hai dạng ảnh chính:

- Ảnh chụp gần, rõ nét, tập trung vào chi tiết hư hỏng.
- Ảnh chụp xa, thể hiện toàn cảnh xe nhưng vùng hư hỏng nhỏ và khó quan sát hơn.

Sự khác biệt này giúp mô hình được tiếp cận với nhiều tình huống đầu vào hơn. Nhóm ảnh cận cảnh hỗ trợ mô hình học hình dạng, kết cấu và đường biên của hư hỏng; trong khi nhóm ảnh toàn cảnh giúp mô hình học cách phát hiện hư hỏng khi vùng hỏng có kích thước nhỏ và xuất hiện trong bối cảnh phức tạp.

Tuy nhiên, phân bố dữ liệu hiện tại nghiêng nhiều hơn về các ảnh có chất lượng tốt và chụp cận cảnh, do hai bộ instance segmentation có tổng số lượng ảnh lớn hơn bộ object detection. Điều này có thể khiến mô hình đạt kết quả tốt hơn trên các ảnh rõ nét, chụp gần, nhưng giảm độ chính xác khi xử lý ảnh chụp xa, ảnh mờ hoặc ảnh có độ phân giải thấp. Đây là một dạng chênh lệch phân bố dữ liệu cần được lưu ý khi đánh giá khả năng tổng quát hóa của mô hình.

Ngoài ra, việc hợp nhất dữ liệu từ các nguồn có kiểu annotation khác nhau cũng có thể làm xuất hiện sự không đồng nhất về cách đánh dấu hư hỏng. Instance segmentation mô tả sát đường biên đối tượng, trong khi object detection chỉ sử dụng bounding box. Trước khi huấn luyện YOLOv8s, các annotation cần được chuyển đổi và chuẩn hóa về cùng định dạng YOLO. Sau quá trình chuyển đổi, vùng segmentation được biểu diễn bằng bounding box bao quanh vùng mask để phù hợp với bài toán object detection của dự án.

Nhìn chung, bộ dữ liệu merge_data có ưu điểm là kết hợp được cả ảnh cận cảnh chất lượng tốt và ảnh toàn cảnh có điều kiện khó hơn. Tuy nhiên, dữ liệu vẫn tồn tại một số hạn chế như chất lượng ảnh không đồng đều, tỷ lệ ảnh chụp gần và chụp xa chưa cân bằng, kiểu annotation ban đầu khác nhau và vùng hư hỏng trong ảnh toàn cảnh thường có kích thước nhỏ. Những yếu tố này có thể ảnh

hưởng đến kết quả Precision, Recall và mAP của mô hình, đặc biệt đối với các lớp có đặc điểm nhỏ hoặc khó quan sát như crack và scratch.

Trong tương lai, dự án có thể bổ sung thêm ảnh chụp toàn cảnh có chất lượng cao, ảnh chụp trong nhiều điều kiện ánh sáng và ảnh có vùng hư hỏng ở nhiều kích thước khác nhau. Việc cân bằng tốt hơn giữa ảnh cận cảnh và ảnh toàn cảnh sẽ giúp mô hình hoạt động ổn định hơn khi được tích hợp vào ứng dụng thực tế.

3.3. Dataset phân loại mức độ hư hỏng

Bên cạnh bộ dữ liệu dùng để phát hiện vị trí và loại hư hỏng bằng YOLOv8s, dự án sử dụng một bộ dữ liệu ảnh riêng cho bài toán phân loại mức độ hư hỏng. Mục tiêu của bộ dữ liệu này là huấn luyện mô hình CNN/ConvNeXt-Tiny nhận ảnh toàn cảnh do người dùng tải lên và phân loại mức độ hư hỏng tổng quát của ảnh thành ba lớp: minor, moderate và severe.

Khác với nhánh YOLO, nhánh severity classification không sử dụng bounding box hoặc ảnh crop từ vùng phát hiện. Đầu vào của mô hình là toàn bộ ảnh xe. Cách tiếp cận này giúp mô hình khai thác cả đặc điểm của vùng hư hỏng và ngữ cảnh tổng thể như số lượng bộ phận bị ảnh hưởng, mức độ biến dạng của thân xe và phạm vi hư hỏng xuất hiện trong ảnh.

3.3.1. Nguồn dữ liệu và mục đích sử dụng

Dữ liệu ảnh cho bài toán severity classification được lấy từ bộ Car Damage Detection trên Kaggle:

<https://www.kaggle.com/datasets/asfarhossainsitab/car-damage-detection>

Các ảnh được tổ chức lại theo ba thư mục tương ứng với ba mức độ hư hỏng là minor, moderate và severe. Mỗi ảnh chỉ được gán một nhãn severity tổng quát, đại diện cho mức độ hư hỏng nổi bật của chiếc xe trong ảnh.

Cần lưu ý rằng bộ dữ liệu severity và bộ dữ liệu YOLO phục vụ hai nhiệm vụ khác nhau. Dataset YOLO cung cấp annotation về vị trí và loại hư hỏng, trong khi dataset severity chỉ cung cấp nhãn mức độ tổng quát của toàn ảnh. Vì vậy, đầu ra của ConvNeXt-Tiny không cho biết chính xác vùng nào thuộc mức độ nặng, mà cung cấp một tín hiệu tổng quan để hỗ trợ tăng điều chỉnh giá.

3.3.2. Tiêu chí phân loại mức độ hư hỏng

Các ảnh được chia thành ba mức độ như sau:

- **Minor – hư hỏng nhẹ:** gồm các hư hỏng có phạm vi nhỏ, ít ảnh hưởng đến hình dạng tổng thể của xe, chẳng hạn vết xước nhẹ, vết lõm nhỏ hoặc hư hỏng bề mặt không làm biến dạng rõ bộ phận xe.



Hình 21: Ví dụ ảnh thuộc lớp minor (hư hỏng nhẹ)

- **Moderate – hư hỏng trung bình:** gồm các trường hợp hư hỏng đã thể hiện rõ trên ảnh, có thể ảnh hưởng đến một hoặc nhiều bộ phận nhưng chưa gây biến dạng nghiêm trọng cho cấu trúc tổng thể. Ví dụ gồm vùng móp tương đối lớn, cản xe bị biến dạng hoặc vết nứt rõ nhưng bộ phận vẫn còn tương đối nguyên vẹn.



Hình 22: Ví dụ ảnh thuộc lớp moderate (hư hỏng trung bình)

- **Severe – hư hỏng nghiêm trọng:** gồm các trường hợp có biến dạng lớn, nứt vỡ rõ ràng, mất một phần bộ phận hoặc nhiều khu vực trên xe bị ảnh hưởng. Các ảnh severe thường thể hiện tai nạn mạnh, cản xe bị vỡ, thân xe biến dạng lớn, kính hoặc đèn bị phá hủy đáng kể.



Hình 23: Ví dụ ảnh thuộc lớp severe (hư hỏng nghiêm trọng)

Ranh giới giữa các lớp không phải lúc nào cũng hoàn toàn rõ ràng. Đặc biệt, lớp moderate nằm giữa hư hỏng nhẹ và nghiêm trọng nên dễ bị nhầm với hai lớp còn lại. Một vết móp có thể được xem là minor hoặc moderate tùy theo diện tích, vị trí, góc chụp và mức độ biến dạng được thể hiện trên ảnh.

Nếu các nhãn severity được gán lại thủ công trong quá trình thực hiện dự án, cần trình bày rõ tiêu chí trên được sử dụng làm quy tắc gán nhãn. Nếu nhãn đã có sẵn trong nguồn dữ liệu, cần ghi rõ dự án giữ nguyên nhãn của nguồn và chỉ tổ chức lại dữ liệu theo định dạng phù hợp với mô hình.

3.3.2. Đặc điểm và chất lượng hình ảnh

Phân bố ảnh theo lớp severity

Dataset CNN severity classification - folder thực tế: 01-minor, 02-moderate, 03-severe

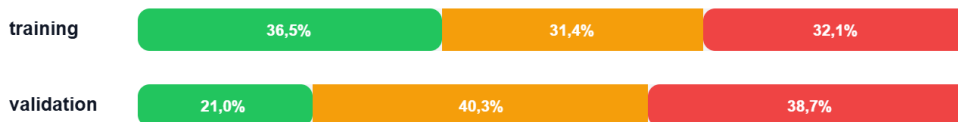
Tổng số ảnh: 3.335

Bảng thống kê

Folder	01-minor	02-moderate	03-severe	Tổng
training	1.076	924	945	2.945
validation	82	157	151	390

Biểu diễn trực quan

01-minor 02-moderate 03-severe



Ghi chú: tập training khá cân bằng giữa ba lớp; tập validation có ít ảnh minor hơn moderate và severe.

Hình 24: Phân bố số lượng ảnh theo ba lớp severity trong tập train và validation

Qua quá trình quan sát dữ liệu, ảnh thuộc ba lớp minor, moderate và severe có sự khác biệt tương đối rõ về khoảng cách chụp, phạm vi khung hình và chất lượng ảnh.

Đối với lớp minor, các hư hỏng thường có kích thước nhỏ và chỉ xuất hiện trên một khu vực giới hạn của xe, chẳng hạn vết xước nhẹ, vết lõm nhỏ hoặc hư hỏng bề mặt. Vì vậy, ảnh của lớp này thường được chụp ở khoảng cách gần và có góc chụp hẹp, tập trung trực tiếp vào vị trí bị hư hỏng. Vùng hỏng thường chiếm tỷ lệ lớn trong khung hình, nhờ đó các chi tiết như đường xước, bề mặt bị lõm hoặc phần sơn bị bong được thể hiện rõ hơn. Chất lượng ảnh của lớp minor nhìn chung cũng tốt hơn, với độ nét cao và ít chi tiết nền không liên quan.



Hình 25: Ví dụ ảnh minor được chụp gần, vùng hư hỏng nhỏ được thể hiện rõ

Ngược lại, các ảnh thuộc lớp moderate và severe thường thể hiện những hư hỏng có phạm vi lớn hơn, có thể ảnh hưởng đến một bộ phận lớn hoặc nhiều khu vực trên xe. Do cần thể hiện tổng thể tình trạng hư hỏng, các ảnh này thường được chụp ở khoảng cách xa hơn và có góc chụp rộng hơn. Trong nhiều trường hợp, phần lớn hoặc toàn bộ chiếc xe xuất hiện trong khung hình để người quan sát có thể đánh giá mức độ biến dạng, số lượng bộ phận bị ảnh hưởng và phạm vi hư hỏng tổng thể.



Hình 26: Ví dụ ảnh moderate/severe được chụp ở góc rộng để thể hiện phạm vi hư hỏng trên xe

Do ảnh moderate và severe thường được chụp toàn cảnh hơn, vùng hư hỏng cụ thể có thể chiếm tỷ lệ nhỏ hơn so với toàn bộ ảnh. Bên cạnh đó, một số ảnh có chất lượng thấp hơn, độ phân giải không cao, bị mờ, nén hoặc chịu ảnh hưởng của điều kiện ánh sáng và bối cảnh xung quanh. Điều này có thể khiến các chi tiết nhỏ của hư hỏng không được thể hiện rõ như trong các ảnh minor chụp cận cảnh

Sự khác biệt này có thể được tóm tắt như sau:

Lớp severity	Đặc điểm hư hỏng	Khoảng cách và góc chụp	Chất lượng ảnh quan sát được
Minor	Hư hỏng nhỏ, tập trung tại một vùng	Thường chụp gần, góc hẹp, tập trung vào vùng hỏng	Thường rõ nét hơn, vùng hỏng chiếm tỷ lệ lớn trong ảnh

Moderate	Hư hỏng rõ, ảnh hưởng một hoặc nhiều bộ phận	Thường chụp xa hơn, góc rộng hơn	Chất lượng không đồng đều, vùng hỏng chiếm tỷ lệ nhỏ hơn
Severe	Hư hỏng lớn, biến dạng mạnh hoặc ảnh hưởng nhiều khu vực	Thường chụp toàn cảnh hoặc phần lớn xe	Một số ảnh mờ hoặc độ phân giải thấp, nhiều chi tiết nền

Sự khác biệt về cách chụp giúp mô hình có thêm thông tin ngữ cảnh để phân biệt các mức độ hư hỏng. Ảnh minor cung cấp nhiều chi tiết cục bộ của vùng hỏng, trong khi ảnh moderate và severe cung cấp thông tin về phạm vi hư hỏng trên tổng thể chiếc xe.

Tuy nhiên, đây cũng là một hạn chế của dataset. Mô hình có thể không chỉ học đặc điểm thực sự của hư hỏng mà còn học các dấu hiệu liên quan đến cách chụp ảnh, chẳng hạn ảnh chụp gần thường thuộc lớp minor, còn ảnh chụp xa và toàn cảnh thường thuộc lớp moderate hoặc severe. Điều này có thể tạo ra sự thiên lệch dữ liệu. Khi gặp một ảnh minor được chụp từ xa hoặc một ảnh severe được chụp cận cảnh, mô hình có thể dự đoán sai do điều kiện ảnh khác với xu hướng phổ biến trong tập huấn luyện.

Vì vậy, trong tương lai cần bổ sung thêm ảnh minor ở góc chụp rộng và ảnh moderate, severe ở góc chụp gần, đồng thời cân bằng chất lượng ảnh giữa các lớp. Điều này sẽ giúp mô hình tập trung nhiều hơn vào đặc điểm của hư hỏng thay vì phụ thuộc vào khoảng cách chụp hoặc độ rõ nét của ảnh.

3.4. Nhận xét về dữ liệu

3.4. Nhận xét về dữ liệu

Trong dự án, ba nhóm dữ liệu được sử dụng cho ba nhiệm vụ riêng biệt và chưa được đồng bộ hoàn toàn theo từng chiếc xe. Dataset dạng bảng được sử dụng để huấn luyện các mô hình Machine Learning như Linear Regression, Random Forest và XGBoost nhằm học mối quan hệ giữa các thông tin kỹ thuật của xe với giá bán. Cột Price trong dataset đóng vai trò là biến mục tiêu của bài toán hồi quy. Vì vậy, nhánh dữ liệu bảng có ground truth để huấn luyện và đánh giá mô hình dự đoán giá cơ bản.

Kết quả đầu ra của mô hình XGBoost là **base price**, tức mức giá ước lượng của xe dựa trên các thuộc tính như năm sản xuất, số km đã đi, loại nhiên liệu, hộp số, dung tích động cơ, công suất và số chủ sở hữu. Giá trị này mới chỉ phản ánh

tình trạng chung của xe theo dữ liệu dạng bảng và chưa xét đến các hư hỏng ngoại thất quan sát được từ hình ảnh.

Hai bộ dữ liệu ảnh được sử dụng độc lập cho nhánh Computer Vision. Dataset object detection được dùng để huấn luyện YOLOv8s phát hiện vị trí, loại và diện tích tương đối của các vùng hư hỏng. Dataset severity classification được dùng để huấn luyện ConvNeXt-Tiny phân loại mức độ hư hỏng tổng quát của ảnh thành minor, moderate hoặc severe. Tuy nhiên, các ảnh hư hỏng này không được ghép với thông tin kỹ thuật và giá giao dịch của chính chiếc xe xuất hiện trong ảnh.

Do đó, dự án **không có ground truth cho giá xe cuối cùng sau khi xét đến tình trạng hư hỏng ngoại thất**. Nói cách khác, không có dữ liệu cho biết một chiếc xe cụ thể có thông số kỹ thuật nhất định, xuất hiện các loại hư hỏng nhất định và được giao dịch thực tế với mức giá bao nhiêu sau khi trừ ảnh hưởng của hư hỏng.

Vì chưa có ground truth cho final price, dự án sử dụng tăng **rule-based adjustment** để kết hợp ba nhóm kết quả:

- Base price do XGBoost dự đoán từ dữ liệu dạng bảng.
- Damage features do YOLOv8s cung cấp, gồm loại hư hỏng, số lượng vùng hỏng, confidence và tỷ lệ diện tích vùng hỏng.
- Severity tổng quát do ConvNeXt-Tiny dự đoán từ ảnh full.

Dựa trên các thông tin trên, hệ thống áp dụng các trọng số và quy tắc được xây dựng trước để tính mức giảm trừ, sau đó xác định giá tham khảo cuối cùng theo công thức tổng quát:

Final price = Base price – Damage adjustment

Cách tiếp cận này giúp dự án minh họa được quy trình kết hợp Machine Learning và Computer Vision trong bài toán định giá xe cũ. Tuy nhiên, đây cũng là một hạn chế quan trọng của hệ thống. Final price không phải là kết quả được mô hình học trực tiếp từ dữ liệu giao dịch thực tế mà phụ thuộc vào các quy tắc và trọng số do người xây dựng hệ thống lựa chọn. Vì vậy, mức giá sau điều chỉnh hiện chỉ mang tính tham khảo, chưa thể được xem là giá trị định giá tuyệt đối hoặc thay thế kết quả thẩm định từ chuyên gia.

Ngoài ra, do chưa có ground truth cho final price nên dự án cũng chưa thể sử dụng các metric hồi quy như MAE, RMSE hoặc R^2 để đánh giá trực tiếp độ chính xác của giá sau điều chỉnh. Các metric hiện tại chỉ đánh giá riêng từng

thành phần, gồm chất lượng dự đoán base price của XGBoost, khả năng phát hiện hư hỏng của YOLOv8s và khả năng phân loại severity của ConvNeXt-Tiny. Sai số của từng thành phần có thể tiếp tục tích lũy và ảnh hưởng đến giá tham khảo cuối cùng.

Trong tương lai, để khắc phục hạn chế này, cần xây dựng một bộ dữ liệu đồng bộ theo từng chiếc xe, bao gồm thông tin kỹ thuật, ảnh ngoại thất, loại và mức độ hư hỏng, chi phí sửa chữa, kết quả thẩm định và giá giao dịch thực tế. Khi có dữ liệu như vậy, hệ thống có thể học trực tiếp mức ảnh hưởng của từng hư hỏng đến giá xe, đồng thời đánh giá final price bằng các chỉ số định lượng thay vì phụ thuộc hoàn toàn vào cơ chế rule-based.

4. Phương pháp xây dựng mô hình

Trong dự án, hệ thống được xây dựng theo hướng pipeline nhiều mô-đun, kết hợp giữa mô hình học máy trên dữ liệu bảng, mô hình phát hiện đối tượng trên ảnh, mô hình phân loại mức độ hư hỏng và tầng suy luận dựa trên luật. Cách thiết kế này phù hợp với bài toán định giá xe cũ vì giá xe không chỉ phụ thuộc vào thông tin kỹ thuật như hãng xe, năm sản xuất, số km đã đi, dung tích động cơ hay hộp số, mà còn chịu ảnh hưởng bởi tình trạng ngoại thất thực tế của xe.

Dữ liệu đầu vào của hệ thống gồm hai nhóm chính. Nhóm thứ nhất là dữ liệu dạng bảng, bao gồm các thông tin mô tả xe. Nhóm dữ liệu này luôn được đưa vào mô hình XGBoost để dự đoán giá cơ sở của xe, gọi là `base price`. Nhóm thứ hai là dữ liệu ảnh, do người dùng có thể upload hoặc không upload. Nếu người dùng không upload ảnh, hệ thống chỉ sử dụng kết quả từ mô hình XGBoost và giá cuối cùng bằng giá cơ sở. Nếu người dùng upload ảnh, ảnh sẽ được đưa vào hai nhánh xử lý song song: YOLOv8s dùng để phát hiện vị trí và loại hư hỏng, còn ConvNeXt-Tiny dùng để phân loại mức độ hư hỏng tổng quát của ảnh.

Sau khi có kết quả từ các mô hình, hệ thống sử dụng tầng rule-based adjustment để điều chỉnh giá. Tầng này kết hợp giá cơ sở từ XGBoost, số lượng hư hỏng, loại hư hỏng, tỷ lệ diện tích vùng hư hỏng và mức độ hư hỏng do CNN dự đoán để tính ra mức khấu trừ tham khảo. Nhờ đó, hệ thống không chỉ đưa ra một con số dự đoán giá, mà còn giải thích được vì sao giá bị điều chỉnh thông qua các thông tin hiển thị trên giao diện như số lượng vùng hư hỏng, loại hư hỏng, diện tích tương đối, mức độ hư hỏng và phần tiền bị trừ.

4.1. Mô hình dự đoán giá từ dữ liệu bảng

Nhánh đầu tiên của hệ thống là mô hình dự đoán giá cơ sở từ dữ liệu bảng. Dữ liệu đầu vào bao gồm các thuộc tính mô tả xe như hãng xe, tên xe, năm sản xuất, số km đã đi, loại nhiên liệu, hộp số, số chủ sở hữu, số chỗ ngồi, dung tích động cơ, công suất cực đại và mức tiêu hao nhiên liệu. Đây là những đặc trưng có ảnh hưởng trực tiếp đến giá trị xe cũ trên thị trường.

Trước khi huấn luyện, dữ liệu được tiền xử lý để đưa về dạng phù hợp cho mô hình học máy. Các biến dạng số được chuẩn hóa hoặc xử lý theo đúng kiểu dữ liệu, còn các biến phân loại như hãng xe, tên xe, loại nhiên liệu và hộp số được mã hóa để mô hình có thể học được quan hệ giữa đặc trưng và giá bán. Ngoài ra, một số đặc trưng mới cũng được xây dựng, ví dụ như tuổi xe, nhằm phản ánh rõ hơn mức độ khấu hao theo thời gian.

Trong giai đoạn thực nghiệm, nhóm đã so sánh nhiều mô hình hồi quy khác nhau như Linear Regression, Random Forest Regressor và XGBoost Regressor. Linear Regression được dùng làm baseline đơn giản để kiểm tra mức độ tuyến tính của bài toán. Random Forest được dùng để thử nghiệm khả năng học quan hệ phi tuyến thông qua tập hợp nhiều cây quyết định. XGBoost được lựa chọn làm mô hình chính vì có khả năng xử lý tốt dữ liệu dạng bảng, học được các quan hệ phi tuyến phức tạp và cho kết quả thực nghiệm tốt hơn các mô hình còn lại.

Đầu ra của mô hình XGBoost là giá cơ sở của xe, tức là giá ước lượng dựa trên thông tin kỹ thuật và thông tin sử dụng, chưa xét đến tình trạng hư hỏng ngoại thất từ ảnh. Giá cơ sở này đóng vai trò nền tảng cho toàn bộ hệ thống. Nếu người dùng không upload ảnh, hệ thống xem như không có thông tin hư hỏng và giá cuối cùng chính là giá cơ sở. Nếu có ảnh, giá cơ sở sẽ tiếp tục được điều chỉnh bởi tầng rule-based dựa trên kết quả phân tích hư hỏng.

4.2. Mô hình phát hiện hư hỏng bằng YOLOv8

Nhánh thứ hai của hệ thống là mô hình phát hiện hư hỏng trên ảnh xe. Đây là bài toán object detection, trong đó mô hình không chỉ cần xác định ảnh có hư hỏng hay không, mà còn phải xác định vị trí cụ thể của từng vùng hư hỏng và phân loại vùng đó thuộc loại nào.

Trong dự án, YOLOv8s được sử dụng để phát hiện các loại hư hỏng ngoại thất như scratch, dent, crack, lamp_broken, glass_broken và tire_flat. Với mỗi vùng hư hỏng được phát hiện, mô hình trả về tọa độ bounding box, nhãn hư

hồng và confidence score. Bounding box cho biết vị trí vùng hư hỏng trên ảnh, nhãn cho biết loại hư hỏng, còn confidence score thể hiện độ tin cậy của mô hình đối với dự đoán đó.

YOLOv8s được lựa chọn vì có sự cân bằng giữa độ chính xác và tốc độ xử lý. So với các biến thể nhỏ hơn, YOLOv8s có khả năng học đặc trưng tốt hơn, đặc biệt với các vùng hư hỏng có hình dạng phức tạp hoặc kích thước nhỏ. So với các biến thể lớn hơn, YOLOv8s nhẹ hơn và phù hợp hơn với mục tiêu tích hợp vào ứng dụng demo bằng Streamlit.

Trong hệ thống, kết quả từ YOLOv8s không chỉ dùng để vẽ bounding box lên ảnh cho người dùng quan sát, mà còn được chuyển thành các đặc trưng phục vụ điều chỉnh giá. Các đặc trưng quan trọng bao gồm số lượng vùng hư hỏng, loại hư hỏng, confidence score và tỷ lệ diện tích vùng hư hỏng so với toàn bộ ảnh. Cụ thể, với mỗi bounding box, hệ thống có thể tính diện tích tương đối theo công thức:

$$\text{area_ratio} = \text{diện tích bounding box} / \text{diện tích toàn ảnh}$$

Giá trị area_ratio giúp hệ thống ước lượng mức độ lan rộng của vùng hư hỏng trên ảnh. Tuy nhiên, giá trị này cũng có hạn chế vì phụ thuộc vào khoảng cách chụp và góc chụp. Cùng một vết hư hỏng, nếu ảnh được chụp gần thì vùng hư hỏng có thể chiếm diện tích lớn hơn, còn nếu ảnh chụp xa thì tỷ lệ diện tích có thể nhỏ hơn. Vì vậy, trong hệ thống hiện tại, diện tích vùng hư hỏng chỉ nên được xem là một tín hiệu hỗ trợ, không phải yếu tố duy nhất quyết định mức giảm giá.

4.3. Mô hình phân loại mức độ hư hỏng bằng ConvNeXt-Tiny

Bên cạnh việc phát hiện vị trí và loại hư hỏng bằng YOLOv8s, hệ thống còn sử dụng mô hình CNN để phân loại mức độ hư hỏng tổng quát của ảnh. Mô hình được lựa chọn trong phiên bản hiện tại là ConvNeXt-Tiny.

Khác với YOLOv8s, đầu vào của ConvNeXt-Tiny không phải là các vùng crop từ bounding box, mà là ảnh full do người dùng upload. Mục tiêu của mô hình là đánh giá tổng thể mức độ hư hỏng của xe trong ảnh và phân loại vào một trong ba nhóm: minor, moderate hoặc severe. Cách xử lý này phù hợp hơn với bài toán thực tế vì mức độ hư hỏng không chỉ phụ thuộc vào một vùng nhỏ, mà còn liên quan đến toàn bộ ngữ cảnh ảnh, số lượng vùng hư hỏng và cảm nhận tổng thể về tình trạng ngoại thất.

ConvNeXt-Tiny được lựa chọn sau khi nhóm thử nghiệm và so sánh với một số mô hình CNN khác như ResNet18, ResNet50, EfficientNet-B0 và EfficientNet-B2. So với ResNet18, ConvNeXt-Tiny có khả năng biểu diễn đặc trưng ảnh tốt hơn, đặc biệt trong các trường hợp vết hư hỏng có hình dạng phức tạp, mức độ hư hỏng không rõ ràng hoặc ảnh có nền phức tạp. Mô hình này cũng phù hợp với hướng fine-tuning từ mô hình đã được huấn luyện trước trên tập dữ liệu lớn.

Đầu ra của ConvNeXt-Tiny gồm nhãn mức độ hư hỏng và xác suất tương ứng cho từng lớp. Ví dụ, mô hình có thể trả về kết quả moderate với xác suất cao nhất, đồng thời hiển thị xác suất của cả ba nhóm minor, moderate và severe. Những xác suất này giúp người dùng hiểu được mức độ chắc chắn của mô hình thay vì chỉ nhận một nhãn kết quả duy nhất.

Trong tầng điều chỉnh giá, severity được quy đổi thành điểm số. Một cách biểu diễn đơn giản là:

minor = 1

moderate = 2

severe = 3

Như vậy, nếu ảnh được phân loại là severe, cùng một loại hư hỏng và cùng số lượng vùng hư hỏng, mức khấu trừ sẽ cao hơn so với trường hợp ảnh được phân loại là minor.

4.4. Cơ chế điều chỉnh giá sau hư hỏng

Sau khi có giá cơ sở từ XGBoost, thông tin vùng hư hỏng từ YOLOv8s và mức độ hư hỏng từ ConvNeXt-Tiny, hệ thống sử dụng tầng rule-based adjustment để tính giá xe sau điều chỉnh. Đây là phần kết nối các mô hình riêng lẻ thành một hệ thống định giá hoàn chỉnh.

Tầng rule-based nhận các đầu vào chính sau:

base_price: giá cơ sở do XGBoost dự đoán

damage_count: số lượng vùng hư hỏng YOLO phát hiện

damage_class: loại hư hỏng của từng vùng

area_ratio: tỷ lệ diện tích vùng hư hỏng so với ảnh

severity: mức độ hư hỏng tổng quát do ConvNeXt-Tiny dự đoán

confidence: độ tin cậy của mô hình với từng dự đoán

Mỗi loại hư hỏng được gán một trọng số tương đối để phản ánh mức độ ảnh hưởng đến giá trị xe. Ví dụ, vết xước nhẹ thường ảnh hưởng ít hơn so với đèn hỏng hoặc kính vỡ vì các hư hỏng liên quan đến đèn, kính và lốp có thể ảnh hưởng trực tiếp đến an toàn vận hành và chi phí thay thế. Có thể thiết lập trọng số tham khảo như sau:

scratch: 0.75

dent: 1.00

crack: 1.15

tire_flat: 1.10

glass_broken: 1.25

lamp_broken: 1.35

Trong đó, dent được chọn làm mốc cơ sở với hệ số 1.0. Các loại hư hỏng nhẹ hơn như scratch có hệ số thấp hơn, còn các hư hỏng liên quan đến an toàn hoặc thay thế linh kiện như lamp_broken và glass_broken có hệ số cao hơn.

Với mỗi vùng hư hỏng, hệ thống có thể tính điểm hư hỏng cục bộ theo công thức tổng quát:

$\text{damage_score}_i = \text{severity_score} \times \text{class_weight} \times \text{area_factor}$

Trong đó, severity_score là điểm mức độ hư hỏng do ConvNeXt-Tiny cung cấp, class_weight là trọng số của loại hư hỏng và area_factor là hệ số phản ánh diện tích tương đối của vùng hư hỏng. Tổng điểm hư hỏng của xe được tính bằng cách cộng điểm của tất cả các vùng hư hỏng được phát hiện:

$\text{total_damage_score} = \text{tổng damage_score}_i$

Sau đó, hệ thống chuyển tổng điểm hư hỏng thành tỷ lệ khấu trừ giá. Để tránh trường hợp giá bị giảm quá mạnh do nhiều bounding box nhỏ hoặc do ảnh chụp quá gần, hệ thống nên giới hạn mức khấu trừ tối đa. Ví dụ:

$\text{deduction_rate} = \min(\text{max_deduction_rate}, \text{total_damage_score} \times \text{scale_factor})$

$\text{damage_deduction} = \text{base_price} \times \text{deduction_rate}$

$\text{final_price} = \text{base_price} - \text{damage_deduction}$

Trong đó, `max_deduction_rate` là mức giảm tối đa cho phép, còn `scale_factor` là hệ số điều chỉnh để đưa điểm hư hỏng về tỷ lệ phần trăm giảm giá. Cách làm này giúp hệ thống ổn định hơn và tránh việc một ảnh có bounding box lớn bất thường làm giá bị giảm quá mức.

Do dự án hiện tại chưa có ground truth trực tiếp cho giá xe sau khi xét hư hỏng, tăng điều chỉnh giá chưa phải là một mô hình học có giám sát. Đây là một cơ chế suy luận dựa trên luật, dùng để minh họa cách kết hợp kết quả của các mô hình học máy vào bài toán định giá thực tế. Vì vậy, kết quả giá sau điều chỉnh mang tính tham khảo và có thể được cải thiện trong tương lai nếu thu thập được dữ liệu thực tế gồm ảnh xe, loại hư hỏng, chi phí sửa chữa và giá giao dịch sau khi xe bị hư hỏng.

4.5. Hiển Thị Kết Quả Và Khả Năng Giải Thích Trên Giao Diện

Một điểm quan trọng của hệ thống là không chỉ trả về giá cuối cùng, mà còn hiển thị các thông tin trung gian để người dùng hiểu được quá trình xử lý. Trên giao diện Streamlit, người dùng nhập thông tin xe vào form dữ liệu bảng. Sau khi nhấn phân tích, hệ thống hiển thị giá cơ sở do XGBoost dự đoán. Nếu người dùng không upload ảnh, phần khấu trừ hư hỏng bằng 0 và giá cuối cùng bằng giá cơ sở.

Tabular pricing, damage detection, severity grading, and price adjustment in one demo.

Input
Provide car details. Upload images if you want damage-aware pricing.

Car Information Form

Brand	Model
Toyota	Other / nhập ngoài
Year	Custom Model
2018	Vios
KM Driven	Number of Seats
45000	5
Transmission	Fuel Type
Automatic	Petrol
Fuel Consumption (km/l)	Owner Count
18,00	1
Max Power	Engine Displacement (cc)
107,00	1500,00

Hình 27: Giao diện nhập thông tin xe trên ứng dụng Streamlit

Khi người dùng upload ảnh, hệ thống hiển thị thêm các kết quả từ nhánh xử lý ảnh. Với YOLOv8s, giao diện có thể hiển thị ảnh đã được vẽ bounding

Thông tin này giúp giải thích vì sao cùng một số lượng hư hỏng nhưng mức giảm giá có thể khác nhau. Một ảnh được đánh giá là severe sẽ làm hệ thống tăng mức khấu trừ so với ảnh được đánh giá là minor.

Severity Results

Severity Analysis

Total Damages	Images Classified	Max Severity	Average Severity Score
1	1	Minor	1.00
Dents	Scratches	Cracks	
1	0	0	

Metric	Value
Total damages	1
Images classified by CNN	1
Num dents	1
Num scratches	0
Num cracks	0
Max severity	minor
Average severity score	1.00

Hình 29: Kết quả phân tích mức độ hư hỏng tổng quát bằng ConvNeXt-Tiny

Ngoài ra, hệ thống còn có thể hiển thị lý do khấu trừ chính, tổng điểm hư hỏng và tóm tắt số lượng hư hỏng. Nhờ đó, kết quả của hệ thống có tính giải thích tốt hơn so với việc chỉ đưa ra một con số giá cuối cùng. Đây là điểm quan trọng trong bài toán định giá xe cũ, vì người dùng cần biết giá bị thay đổi do yếu tố nào: thông tin xe, số lượng hư hỏng, loại hư hỏng, diện tích vùng hư hỏng hay mức độ nghiêm trọng tổng quát.

Cuối cùng, giao diện hiển thị ba giá trị chính trong phần kết quả định giá:

Severity Details by Full Image

image_name	severity	severity_score	confidence	minor_prob	moderate_prob	severe_prob
002330.jpg	minor	1	63.70%	63.70%	35.90%	0.40%

Pricing Results

Base Price	Damage Deduction	Final Adjusted Price
341,200,000 VND	4,400,000 VND	336,800,000 VND

Pricing Explanation

Top deduction reason: 1 dent damage(s), max severity minor

Total damage score: 1.3%

Metric	Value
Number of damages	1
Highest severity	minor
Estimated deduction rate	1.3%
Final adjusted price	336,800,000 VND

Hình 30: Kết quả điều chỉnh giá và giải thích mức khấu trừ trên giao diện Streamlit

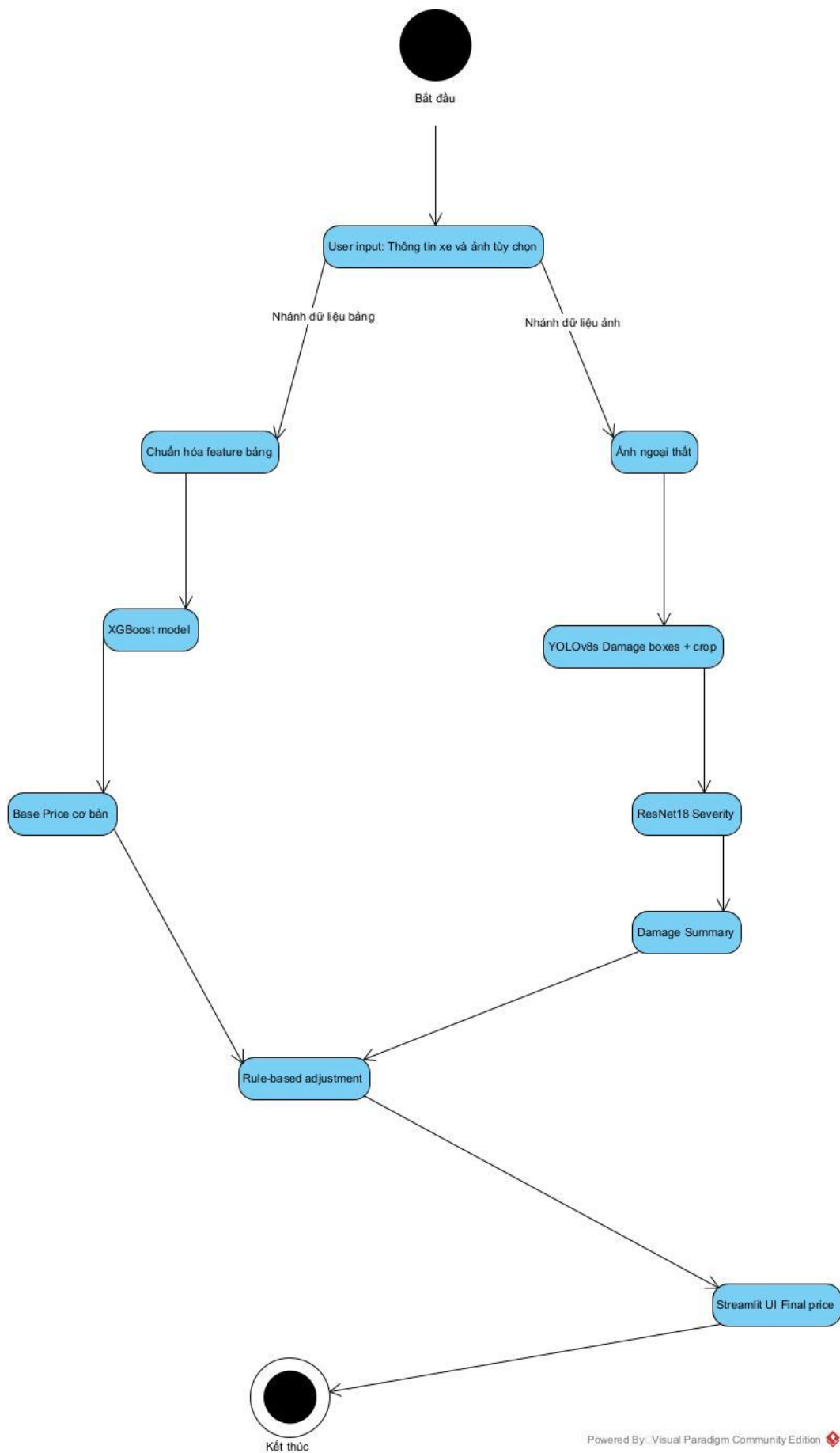
5. Thiết kế hệ thống

5.1. Kiến trúc tổng thể

Hệ thống được thiết kế theo kiến trúc pipeline nhiều mô-đun, trong đó mỗi mô-đun đảm nhiệm một nhiệm vụ riêng trong quá trình định giá xe ô tô cũ. Thay vì sử dụng một mô hình duy nhất cho toàn bộ bài toán, dự án tách hệ thống thành các thành phần nhỏ hơn gồm: mô-đun giao diện người dùng, mô-đun dự đoán giá cơ bản, mô-đun phát hiện hư hỏng, mô-đun phân loại mức độ hư hỏng và mô-đun điều chỉnh giá cuối cùng.

Cách thiết kế này phù hợp với bài toán của dự án vì dữ liệu đầu vào bao gồm cả dữ liệu dạng bảng và dữ liệu ảnh. Dữ liệu dạng bảng được dùng để dự đoán giá cơ bản của xe. Dữ liệu ảnh, nếu được người dùng upload, sẽ được đưa song song vào YOLOv8s để phát hiện hư hỏng và ConvNeXt-Tiny để phân loại mức độ hư hỏng tổng quát từ ảnh full.

Về tổng thể, hệ thống hoạt động theo luồng: người dùng nhập thông tin xe và có thể tải ảnh lên giao diện Streamlit. Thông tin xe luôn được đưa vào XGBoost để dự đoán base price. Nếu người dùng có tải ảnh, ảnh được đưa song song vào YOLOv8s để phát hiện hư hỏng và ConvNeXt-Tiny để phân loại severity từ ảnh full. Cuối cùng, hệ thống kết hợp base price, damage features và severity để tính giá tham khảo sau điều chỉnh; nếu không có ảnh, final price bằng base price từ XGBoost.



Hình 31: Sơ đồ pipeline kiến trúc tổng thể của hệ thống

Kiến trúc này giúp hệ thống có tính mô-đun cao. Mỗi thành phần có thể được kiểm thử, thay thế hoặc cải thiện độc lập. Ví dụ, mô hình YOLO có thể được thay bằng checkpoint tốt hơn trong tương lai mà không cần thay đổi toàn bộ hệ thống. Tương tự, mô hình dự đoán giá hoặc công thức điều chỉnh giá cũng có thể được cải tiến riêng.

Mô tả kiến trúc tổng thể của hệ thống:

Đầu vào người dùng: Hiển thị rõ ràng các bước nhập liệu dữ liệu bảng và tải ảnh thông qua giao diện Streamlit.

Luồng xử lý dữ liệu song song:

- **Dữ liệu bảng:** Chuyển thẳng đến mô hình dự đoán giá cơ bản.
- **Dữ liệu hình ảnh:** Được đưa song song vào hai nhánh. Nhánh YOLOv8s phát hiện hư hỏng và trả về bounding box, class, confidence, area ratio. Nhánh ConvNeXt-Tiny nhận ảnh full và trả về severity tổng quát.

Tổng hợp dữ liệu: Kết quả từ nhánh hình ảnh gồm damage features từ YOLOv8s và severity từ ConvNeXt-Tiny được tổng hợp lại.

Điều chỉnh giá Rule-based: Kết hợp base price từ XGBoost, damage features và severity để đưa ra mức điều chỉnh cuối cùng.

Hiển thị kết quả: Giao diện Streamlit hiển thị mức giá đã điều chỉnh và chi tiết các hư hỏng được phát hiện.

5.2. Luồng xử lý

Luồng xử lý của hệ thống bắt đầu từ việc người dùng truy cập giao diện ứng dụng. Tại đây, người dùng nhập các thông tin cơ bản của xe như hãng xe, dòng xe, năm sản xuất, số km đã đi, loại nhiên liệu, hộp số, số chủ sở hữu, số chỗ ngồi, dung tích động cơ, công suất và mức tiêu hao nhiên liệu. Người dùng có thể tải lên một hoặc nhiều ảnh chụp ngoại thất nếu muốn hệ thống phân tích thêm tình trạng hư hỏng.

Sau khi người dùng nhấn nút phân tích, hệ thống sẽ kiểm tra dữ liệu đầu vào và bắt đầu chạy pipeline dự đoán. Đầu tiên, thông tin dạng bảng của xe được chuẩn hóa về đúng định dạng feature mà mô hình tabular đã được huấn luyện. Các thông tin này sau đó được đưa vào mô hình hồi quy để dự đoán giá cơ bản

của xe. Giá cơ bản này phản ánh giá trị ước lượng của xe dựa trên các thuộc tính kỹ thuật, chưa xét đến tình trạng hư hỏng ngoại thất.

Nếu người dùng upload ảnh, ảnh xe được đưa song song vào hai nhánh xử lý. Nhánh YOLOv8s phát hiện các vùng hư hỏng, trả về danh sách bounding box, nhãn loại hư hỏng, độ tin cậy và tỷ lệ diện tích vùng hư hỏng so với toàn bộ ảnh. Nhánh ConvNeXt-Tiny nhận ảnh full và dự đoán mức độ hư hỏng tổng quát của ảnh thành minor, moderate hoặc severe.

Sau khi có kết quả phát hiện hư hỏng, hệ thống vẽ bounding box lên ảnh để tạo ảnh kết quả trực quan cho người dùng. Kết quả severity từ ConvNeXt-Tiny được hiển thị riêng như một đánh giá tổng quát của ảnh full, không phải kết quả phân loại từng vùng cắt từ YOLO.

Tiếp theo, hệ thống tổng hợp kết quả từ hai nhánh ảnh. Các thông tin được tổng hợp gồm tổng số hư hỏng, số lượng từng loại hư hỏng, area ratio từ YOLOv8s và severity tổng quát từ ConvNeXt-Tiny. Những thông tin này được kết hợp với giá cơ bản để tính toán mức giảm giá tham khảo.

Cuối cùng, hệ thống hiển thị kết quả lên giao diện, bao gồm giá cơ bản, số tiền giảm trừ và giá xe tham khảo sau điều chỉnh. Nếu có ảnh đầu vào, hệ thống hiển thị thêm ảnh gốc, ảnh có bounding box, bảng danh sách hư hỏng phát hiện được và kết quả phân loại severity tổng quát. Nếu không có ảnh, hệ thống bỏ qua nhánh YOLO/CNN và giá tham khảo cuối cùng bằng giá dự đoán từ XGBoost.

Tóm tắt luồng xử lý:

- Bước 1: Người dùng nhập thông tin xe và có thể tải ảnh xe lên hệ thống.
- Bước 2: Hệ thống chuẩn hóa thông tin xe thành các feature đầu vào.
- Bước 3: Mô hình tabular dự đoán giá cơ bản của xe.
- Bước 4: Nếu có ảnh, YOLOv8s phát hiện các vùng hư hỏng trên ảnh xe và trích xuất damage features.
- Bước 5: Đồng thời, ConvNeXt-Tiny nhận ảnh full để phân loại severity tổng quát.
- Bước 6: Hệ thống vẽ bounding box và tổng hợp kết quả YOLOv8s với severity từ ConvNeXt-Tiny.
- Bước 7: Rule-based adjustment tính giá tham khảo sau điều chỉnh từ base price, damage features và severity.
- Bước 8: Giao diện hiển thị kết quả cuối cùng cho người dùng.
- Bước 9: Nếu người dùng không upload ảnh, hệ thống chỉ hiển thị base price và final price bằng base price.

5.3. Các module chính

Hệ thống được tổ chức thành nhiều module nhằm tách biệt rõ ràng giữa giao diện, xử lý mô hình, tiện ích và dữ liệu. Cách tổ chức này giúp mã nguồn dễ quản lý, dễ kiểm thử và dễ mở rộng.

Module đầu tiên là giao diện người dùng, được xây dựng bằng Streamlit trong file `app.py` và các file thuộc thư mục `giao_dien/`. Module này chịu trách nhiệm hiển thị form nhập thông tin xe, khu vực tải ảnh, ảnh xem trước, bảng kết quả phát hiện hư hỏng, kết quả phân loại mức độ và kết quả định giá. Đây là điểm tương tác chính giữa người dùng và hệ thống.

Module thứ hai là mô-đun định giá, nằm trong `dich_vu/dinh_gia.py`. Module này chịu trách nhiệm dự đoán giá cơ bản từ mô hình tabular đã huấn luyện và tính toán giá tham khảo sau điều chỉnh. Đầu vào của module là thông tin xe đã được chuẩn hóa và kết quả hư hỏng từ các mô hình ảnh. Đầu ra là giá cơ bản, mức giảm trừ và giá tham khảo cuối cùng.

Module thứ ba là mô-đun phát hiện hư hỏng, nằm trong `dich_vu/phat_hien_hu_hong.py`. Module này sử dụng mô hình YOLOv8 để phát hiện các vùng hư hỏng trên ảnh xe. Kết quả trả về gồm mã hư hỏng, tên ảnh, loại hư hỏng, độ tin cậy, tọa độ bounding box và tỷ lệ diện tích vùng hỏng. Module này cũng có chức năng vẽ bounding box lên ảnh để phục vụ hiển thị.

Module thứ tư là mô-đun phân loại mức độ hư hỏng, nằm trong `dich_vu/muc_do_hu_hong.py`. Module này sử dụng mô hình CNN/ConvNeXt-Tiny để phân loại severity tổng quát từ ảnh full do người dùng upload. Đầu ra là nhãn minor, moderate hoặc severe kèm xác suất dự đoán. Kết quả này được dùng để hiển thị trên giao diện và hỗ trợ tăng điều chỉnh giá.

Module thứ năm là mô-đun tiện ích, nằm trong thư mục `tien_ich/`. Module này chứa các hàm hỗ trợ như định dạng tiền tệ, định dạng phần trăm, dữ liệu mẫu và quản lý trạng thái phiên làm việc của Streamlit. Việc tách các tiện ích này ra riêng giúp code chính ngắn gọn hơn và tránh lặp lại logic.

Module thứ sáu là mô-đun huấn luyện và xử lý dữ liệu, nằm trong thư mục `src/`. Thư mục này chứa các file phục vụ quá trình load dữ liệu, làm sạch dữ liệu, feature engineering, tiền xử lý, huấn luyện và đánh giá mô hình tabular. Đây là phần dùng trong giai đoạn phát triển mô hình, không trực tiếp hiển thị trên giao diện nhưng đóng vai trò quan trọng trong việc tạo ra model phục vụ hệ thống.

Các module chính trong dự án:

app.py

Điều phối luồng xử lý chính của ứng dụng Streamlit.

giao_dien/

Xây dựng các thành phần giao diện và hiển thị kết quả.

dich_vu/dinh_gia.py

Dự đoán giá cơ bản và tính giá tham khảo sau điều chỉnh.

dich_vu/phat_hien_hu_hong.py

Phát hiện vùng hư hỏng bằng YOLOv8 và vẽ bounding box.

dich_vu/muc_do_hu_hong.py

Phân loại mức độ hư hỏng tổng quát từ ảnh full bằng CNN/ConvNeXt-Tiny.

tien_ich/

Chứa hàm định dạng, dữ liệu mẫu và quản lý session state.

src/

Chứa pipeline xử lý dữ liệu, huấn luyện và đánh giá mô hình.

tests/

Chứa file các file test, minitest để kiểm tra các mô hình và phân bố dữ liệu

Models/

Lưu các model đã huấn luyện như model tabular, YOLO và CNN.

Datasets/

Lưu dữ liệu dạng bảng và dữ liệu phục vụ huấn luyện.

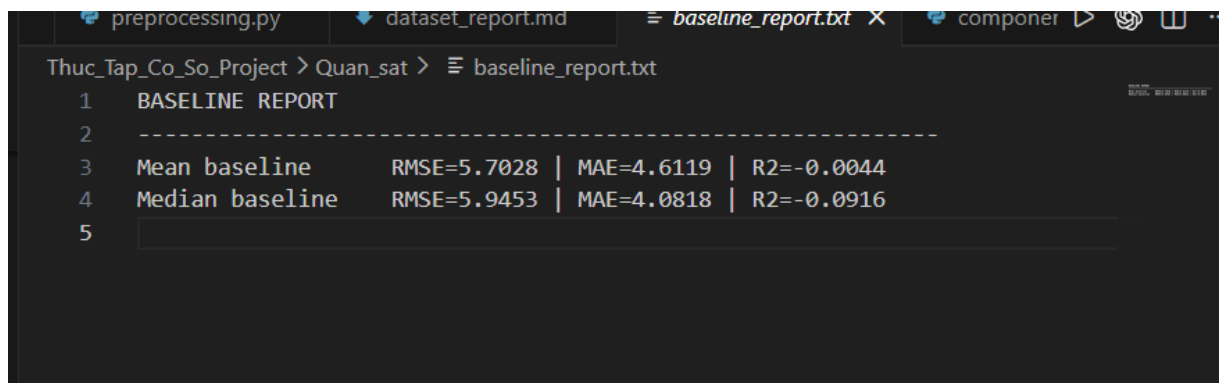
6. Kết quả thực nghiệm

6.1. Kết quả mô hình dự đoán giá từ dữ liệu bảng

Trong phần thực nghiệm với dữ liệu dạng bảng, dự án sử dụng tập dữ liệu xe ô tô cũ gồm 6019 dòng và 14 cột ban đầu. Sau quá trình làm sạch dữ liệu, loại bỏ các dòng không hợp lệ và xử lý các giá trị thiếu, tập train được chia thành 4780 mẫu huấn luyện và 1196 mẫu validation. Ngoài ra, tập test gồm 1223 mẫu được dùng để sinh dự đoán sau khi mô hình đã được huấn luyện.

Các đặc trưng đầu vào sau quá trình tiền xử lý gồm 11 đặc trưng số và 2 đặc trưng phân loại. Nhóm đặc trưng số bao gồm loại nhiên liệu, hộp số, số chủ sở hữu, mức tiêu hao nhiên liệu, dung tích động cơ, công suất, số chỗ ngồi, tuổi xe, số km trung bình mỗi năm, biến đánh dấu xe chạy nhiều và log số km đã đi. Hai đặc trưng phân loại là hãng xe và nhóm tên xe phổ biến.

Trước khi huấn luyện mô hình, dự án xây dựng baseline đơn giản bằng cách dự đoán theo giá trị trung bình và trung vị của tập train. Kết quả baseline như sau:



```
preprocessing.py dataset_report.md baseline_report.txt componer
Thuc_Tap_Co_So_Project > Quan_sat > baseline_report.txt
1  BASELINE REPORT
2  -----
3  Mean baseline      RMSE=5.7028 | MAE=4.6119 | R2=-0.0044
4  Median baseline    RMSE=5.9453 | MAE=4.0818 | R2=-0.0916
5
```

Hình 32: Kết quả baseline trên dữ liệu giá xe cũ

Sau đó, dự án huấn luyện và so sánh ba mô hình hồi quy gồm Linear Regression, Random Forest và XGBoost. Kết quả trên tập validation như sau:

Mô hình	RMSE	MAE	R ²
Linear Regression	2.5325	1.8572	0.8019
Random Forest	1.6074	0.9962	0.9202

Mô hình	RMSE	MAE	R ²
XGBoost	1.4313	0.8914	0.9367

Các giá trị RMSE và MAE được tính theo cùng đơn vị với biến mục tiêu trong dataset. Trong bộ dữ liệu xe cũ sử dụng cho dự án, giá xe được biểu diễn theo đơn vị lakh, vì vậy MAE = 0.8914 có thể hiểu là sai số tuyệt đối trung bình khoảng 0.8914 lakh trên tập validation.

Kết quả cho thấy cả ba mô hình đều tốt hơn rõ rệt so với baseline. Trong đó, XGBoost đạt kết quả tốt nhất với RMSE = 1.4313, MAE = 0.8914 và R² = 0.9367. Điều này cho thấy mô hình XGBoost học tốt quan hệ phi tuyến giữa các đặc trưng của xe và giá bán.

Dự án cũng thực hiện kiểm tra overfitting bằng 5-fold cross-validation. Kết quả cho thấy XGBoost có R² trung bình CV đạt 0.9417, R² train đạt 0.9897 và khoảng cách train - CV là 0.0480. Theo tiêu chí đánh giá trong dự án, mô hình XGBoost chưa có dấu hiệu overfit rõ rệt và được chọn làm mô hình chính cho nhánh dự đoán giá cơ bản.

Used Car Price AI Demo
Tabular pricing, damage detection, severity grading, and price adjustment in one demo.

Input
Provide car details. Upload images if you want damage aware pricing.

Car Information Form

Brand	Toyota	Model	Innova 2.0 E
Year	2018	Number of Seats	5
KM Driven	45000	Fuel Type	Petrol
Transmission	Automatic	Owner Count	1
Fuel Consumption (km/l)	18.00	Engine Displacement (cc)	1500.00
Max Power	107.00		

Upload car images (optional)
Drag and drop files here
Limit 200MB per file • PNG, JPG, JPEG

Upload images to see a preview.

Pricing Results

Base Price	Damage Deduction	Final Adjusted Price
341,200,000 VND	0 VND	341,200,000 VND

Pricing Explanation
Top deduction reason: No detected damages
Total damage score: 0.0%

Hình 33: Giao diện demo khi nhập thông tin xe

Pricing Results

Base Price	Damage Deduction	Final Adjusted Price
341,200,000 VND	0 VND	341,200,000 VND

Pricing Explanation
Top deduction reason: No detected damages
Total damage score: 0.0%

Hình 34: Kết quả dự đoán giá cơ bản khi chưa tải ảnh hư hỏng

Trong trường hợp người dùng chỉ nhập thông tin xe và không tải ảnh hư hỏng lên hệ thống, ứng dụng hoạt động như một mô hình dự đoán giá baseline

dựa hoàn toàn trên dữ liệu bảng. Với ví dụ xe Toyota Innova 2.0 E sản xuất năm 2018, đã đi 45.000 km, sử dụng xăng, hộp số tự động, 5 chỗ ngồi, dung tích 1500cc và công suất 107 mã lực, mô hình XGBoost dự đoán giá cơ bản của xe là **341.200.000 VND**. Do không có ảnh đầu vào nên hệ thống không thực hiện bước phát hiện hư hỏng bằng YOLO và không có thông tin phân loại mức độ hư hỏng bằng ConvNeXt-Tiny. Vì vậy, mức giảm trừ do hư hỏng bằng **0 VND**, tổng điểm hư hỏng là **0.0%**, và giá cuối cùng sau điều chỉnh vẫn giữ nguyên là **341.200.000 VND**. Kết quả này minh họa vai trò của nhánh tabular baseline trong hệ thống: cung cấp giá tham chiếu ban đầu của xe trước khi xét đến các yếu tố hư hỏng ngoại thất từ ảnh.

6.2. Kết quả thực nghiệm mô hình YOLO

6.2.1. Kết quả huấn luyện trên các cấu hình nền tảng

Mô hình nhận diện hư hỏng vỏ xe được thử nghiệm qua nhiều cấu hình kiến trúc của YOLOv8, bao gồm YOLOv8n, YOLOv8s và YOLOv8m. Sau giai đoạn benchmark ban đầu trên dataset khoảng 4000 ảnh, cấu hình YOLOv8s được giữ lại vì cân bằng tốt giữa tốc độ suy luận và độ chính xác. Từ baseline này, dự án tiếp tục huấn luyện phiên bản merge_data với cùng cấu hình nhưng mở rộng dữ liệu cho các lớp khó.

=== BẢNG SO SÁNH CÁC MÔ HÌNH ===

TÊN MODEL	PRECISION	RECALL	mAP50	mAP50-95
best_100_800sz.pt	0.7563	0.6772	0.7172	0.5494
best_m.pt	0.7724	0.6799	0.7217	0.5783
best_merge_data.pt	0.7272	0.6868	0.7016	0.5522
best_merge_data_1061.pt	0.7528	0.6573	0.6995	0.5491
best_n.pt	0.7630	0.6502	0.6988	0.5505
best_s_100.pt	0.7676	0.6554	0.7023	0.5431
best_s_70.pt	0.7312	0.6640	0.7010	0.5501
best_s_finetune.pt	0.7625	0.6841	0.7072	0.5567
best_v2.pt	0.7491	0.6850	0.7171	0.5620
best_89.pt	0.7267	0.6987	0.7158	0.5675

Hình 35: Kết quả benchmark các checkpoint YOLOv8

6.2.2. Tác động của dataset merge_data

Khác với các run cũ, mô hình YOLO chính hiện tại không còn là s_89 trên dataset gốc. Dự án giữ nguyên cấu hình YOLOv8s, epochs = 100, batch size = 16 và image size = 640, nhưng thay đổi dữ liệu huấn luyện bằng phiên bản merge_data. Dataset ban đầu có khoảng 4000 ảnh, lấy từ bộ Car Damage Detection trên Kaggle và đã được chuyển từ định dạng COCO sang YOLOv8 thông qua Roboflow. Sau đó, dự án bổ sung thêm 2307

ảnh từ hai nguồn Roboflow Universe cho ba lớp yếu nhất là crack, scratch và dent, nâng tổng số ảnh lên khoảng 6307 ảnh.

Điểm quan trọng của lần thử nghiệm này là dữ liệu mới không được thêm đều cho tất cả nhãn. Việc bổ sung tập trung vào các lớp có ranh giới thị giác khó, vùng hổng nhỏ và dễ bị bỏ sót. Vì vậy, mục tiêu của merge_data không chỉ là tăng mAP tổng thể, mà còn cải thiện khả năng nhận diện các hư hỏng khó trong tình huống thực tế.

1. Run s_89 được giữ làm baseline tham khảo cho dataset gốc. Trong khi đó, mô hình merge_data được dùng làm mô hình chính vì có thêm dữ liệu học cho các dạng hư hỏng mà hệ thống cần ưu tiên phát hiện như vết xước, vết nứt và vết móp.
2. Trong report mới tại thư mục Quan_sat/yolo_car_report, mô hình YOLOv8s trên merge_data đạt mAP@0.5 tốt nhất ở epoch 79, với Precision khoảng 0.8005, Recall khoảng 0.6820, mAP@0.5 khoảng 0.7256 và mAP@0.5:0.95 khoảng 0.5684. Nếu xét riêng mAP@0.5:0.95, epoch 75 đạt giá trị cao nhất khoảng 0.5719.

6.2.3. Quyết định lựa chọn mô hình cốt lõi merge_data

Việc lựa chọn merge_data dựa trên hai yếu tố chính. Thứ nhất, mô hình vẫn giữ được hiệu năng validation ổn định với cấu hình train giống baseline. Thứ hai, phiên bản dữ liệu mới phù hợp hơn với mục tiêu thực tế của dự án vì tăng số lượng mẫu cho các lớp khó nhất thay vì chỉ mở rộng dữ liệu một cách ngẫu nhiên.

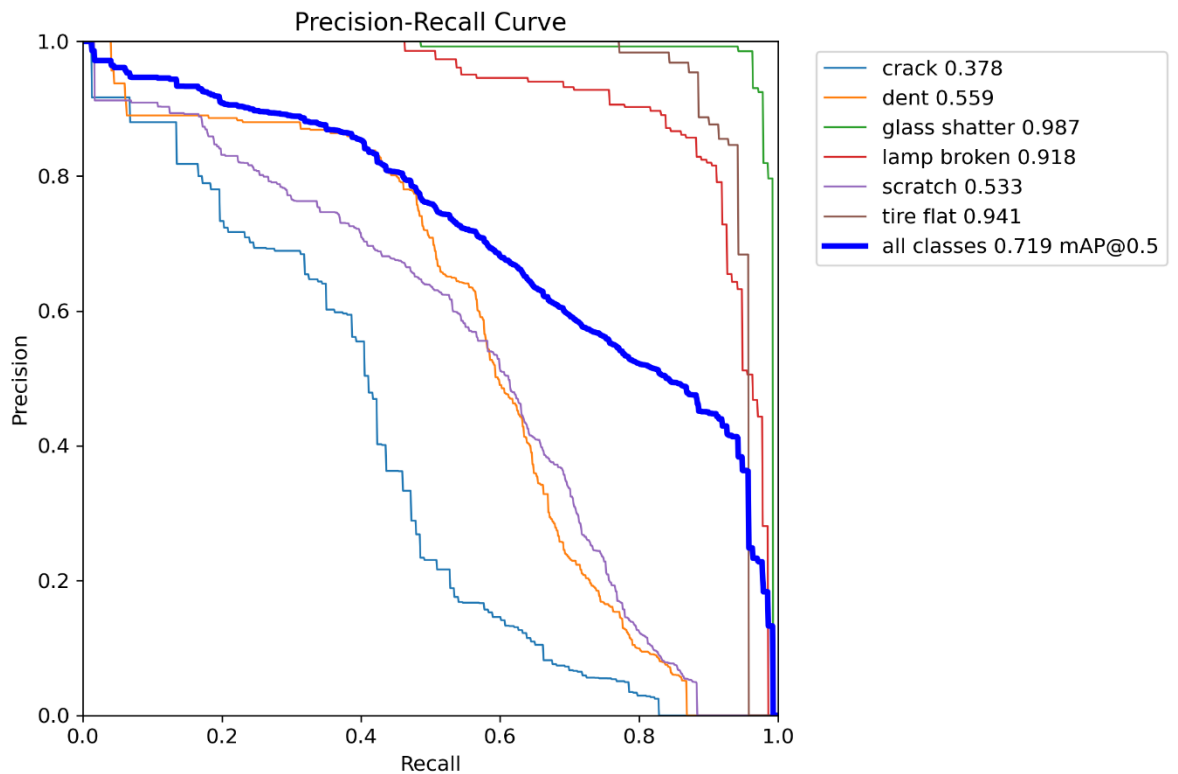
Ba phân lớp scratch, dent và crack ngoài thực tế có ranh giới thị giác mờ, kích thước nhỏ và dễ bị ảnh hưởng bởi ánh sáng, màu sơn hoặc góc chụp. Do đó, việc thêm 2307 ảnh cho riêng ba lớp này giúp mô hình có thêm ví dụ học đúng trọng tâm. Tuy nhiên, vì dữ liệu không được tăng đều cho mọi nhãn, khi phân tích kết quả cần xem xét cả metric tổng thể lẫn mục tiêu giảm bỏ sót các lớp hư hỏng khó.

Bên cạnh các chỉ số validation, khi đánh giá mô hình cũng cần xem xét chất lượng annotation của tập kiểm thử. Một nguyên nhân có thể ảnh hưởng đến metric là một số hư hỏng nhỏ như scratch, dent hoặc crack chưa được gán nhãn đầy đủ trong tập test. Trong trường hợp đó, một số dự đoán hợp lý của mô hình có thể bị hệ thống đánh giá tự động tính thành False Positive, làm giảm Precision hoặc mAP. Vì vậy, ngoài metric định lượng, dự án cũng quan sát thêm kết quả inference trực quan trên ảnh và video thực tế.

Thực nghiệm inference trên video và ảnh thực tế cho thấy mô hình YOLOv8s trên merge_data bám vết ổn định hơn, phát hiện tốt hơn các hư hỏng nhỏ thuộc nhóm scratch, dent và crack. Vì vậy, việc lựa chọn mô hình chính không chỉ dựa trên một con số mAP tĩnh, mà còn dựa trên khả năng tổng quát hóa trong tình huống sử dụng thực tế của hệ thống định giá xe.

Vì mục tiêu của dự án là xây dựng hệ thống định giá có khả năng nhận diện hư hỏng ngoại thất trong thực tế, em quyết định lựa chọn trọng số tốt nhất best.pt của cấu hình YOLOv8s trên merge_data làm mô hình thị giác máy tính chính để tích hợp vào module định giá xe tổng thể.

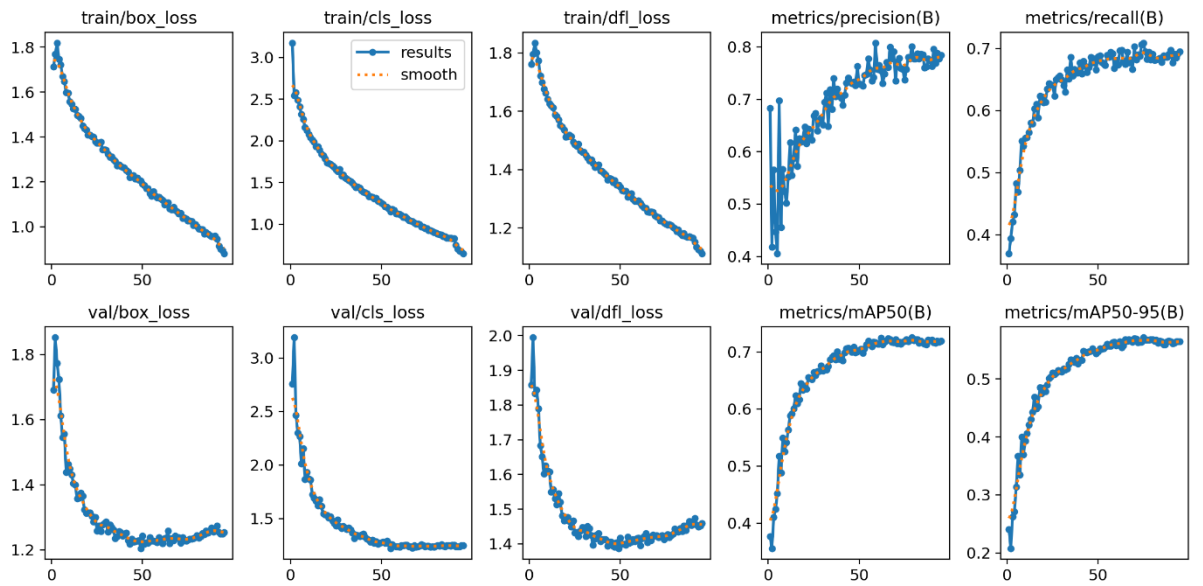
Tham số	Giá trị
Model	YOLOv8s
Epochs	100
Batch size	16
Image size	640
Optimizer	auto
Pretrained	True
Dataset	coco_damage.yolov8 / merge_data
Patience	20



Hình 36: Đường Precision-Recall của mô hình YOLOv8s trên merge_data

Trong report mới, epoch có mAP@0.5 cao nhất là epoch 79 với mAP@0.5 khoảng 0.7256. Ở các epoch cuối, mAP@0.5 vẫn dao động quanh 0.715-0.720 và mAP@0.5:0.95 quanh 0.562-0.565, cho thấy quá trình huấn luyện tương đối ổn định và không bị sụt giảm mạnh ở giai đoạn cuối.

Các chỉ số trong thư mục Quan_sat/yolo_car_report được lấy từ quá trình validation của YOLO. Nếu cần một con số test độc lập sau cùng, cần chạy đánh giá riêng bằng `model.val(split="test")` trên checkpoint đã chốt.



Hình 37: Đường loss và các chỉ số đánh giá trong quá trình huấn luyện YOLOv8s
 Kết quả dự đoán so với nhãn thật được minh họa ở các hình dưới đây.



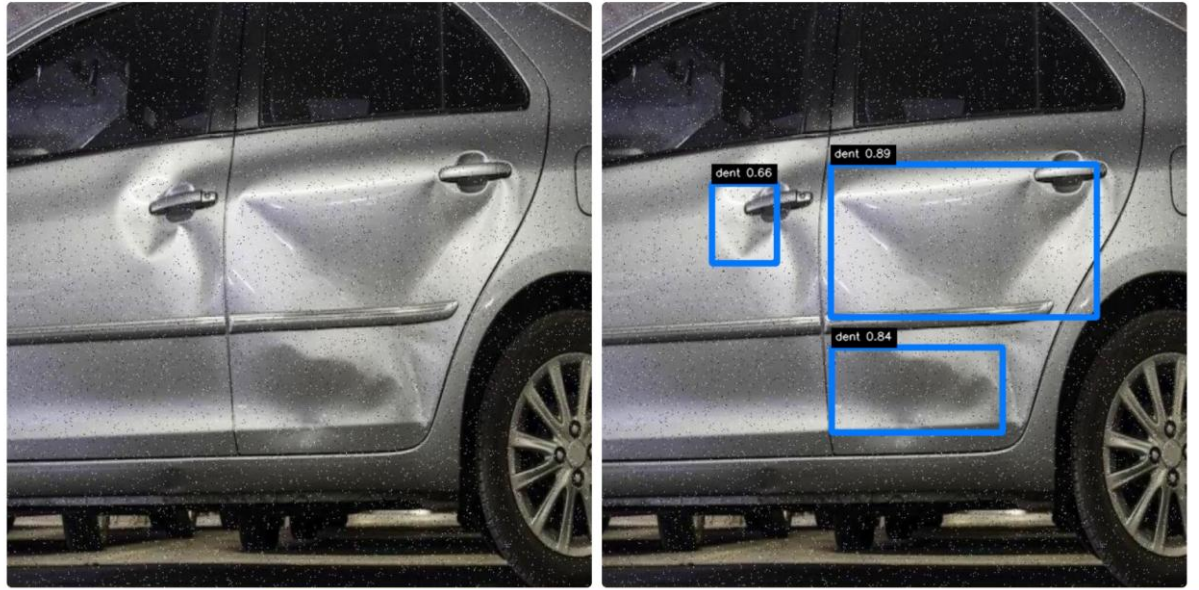
Hình 38: Kết quả dự đoán của YOLOv8s trên một số ảnh validation



Hình 39: Nhận thật của các ảnh validation tương ứng

Kết quả thực nghiệm cho thấy YOLOv8s trên merge_data có khả năng phát hiện tương đối tốt các hư hỏng có hình dạng rõ ràng như kính vỡ, đèn hỏng hoặc lốp xẹp, đồng thời được bổ sung thêm dữ liệu cho các lớp khó như vết nứt, vết móp và vết xước. Dù vậy, các lớp này vẫn là thách thức của bài toán vì vùng hỏng nhỏ, dễ bị ảnh hưởng bởi ánh sáng, góc chụp, màu sơn xe và chất lượng annotation. Trong phạm vi dự án, checkpoint YOLOv8s merge_data được lựa chọn vì đạt mức cân bằng tốt giữa độ chính xác, tốc độ suy luận và khả năng tích hợp vào pipeline demo.

So sánh output của 2 mô hình YOLOv8s_89 và YOLOv8s_merge_data:



Hình 40: Kết quả phát hiện hư hỏng của YOLOv8s_merge_data



Hình 41: Kết quả phát hiện hư hỏng của YOLOv8s_89

Quan sát kết quả trực quan cho thấy mô hình YOLOv8s trên merge_data phát hiện được nhiều vùng hư hỏng hơn so với phiên bản YOLOv8s_89 trên dataset gốc, đặc biệt với các vết lõm (dent). Ở một số ảnh, mô hình cũ chỉ xác định được một vùng nhỏ, trong khi phiên bản merge_data cho kết quả bao quát hơn. Điều này phù hợp với mục tiêu bổ sung dữ liệu cho các lớp hư hỏng khó như dent, scratch và crack.

6.2.4. Nhận xét so sánh với Faster R-CNN

Bên cạnh YOLOv8, Faster R-CNN được xem xét như một hướng tham khảo thuộc nhóm two-stage detector. Khác với YOLO, Faster R-CNN thực hiện phát hiện đối tượng qua hai giai đoạn: đầu tiên sinh các vùng đề xuất có khả năng chứa đối tượng, sau đó phân loại và tinh chỉnh bounding box cho từng vùng. Cách tiếp cận này thường có ưu điểm về độ chính xác trong một số bài toán phát hiện đối tượng, đặc biệt khi cần định vị chi tiết.

Tuy nhiên, với mục tiêu của dự án là xây dựng một hệ thống demo có khả năng phát hiện hư hỏng xe và tích hợp vào pipeline định giá, YOLOv8 phù hợp hơn. YOLO là mô hình one-stage detector, có tốc độ suy luận nhanh hơn, dễ huấn luyện, dễ triển khai và được thư viện Ultralytics hỗ trợ tốt. Điều này giúp quá trình tích hợp vào ứng dụng Streamlit thuận tiện hơn so với Faster R-CNN.

Trong bài toán phát hiện hư hỏng xe, hệ thống không chỉ cần độ chính xác mà còn cần khả năng xử lý nhanh để người dùng có thể nhận kết quả gần như ngay sau khi tải ảnh lên. Faster R-CNN có thể cho kết quả tốt trong một số trường hợp, nhưng chi phí tính toán cao hơn và tốc độ suy luận thường chậm hơn. Vì vậy, mô hình này được xem như một hướng tham khảo để so sánh, trong khi YOLOv8s vẫn được chọn làm mô hình chính của hệ thống.

model	class_id_frcnn	class_id_yolo_equiv	class_name	ap_50_95	ar_100
Faster R-CNN ResNet	1	0	crack	0.1204177886	0.3791410923
Faster R-CNN ResNet	2	1	dent	0.1190894246	0.3319105804
Faster R-CNN ResNet	3	2	glass shatter	0.06109387428	0.2283464521
Faster R-CNN ResNet	4	3	lamp broken	0.1040362641	0.2828358114
Faster R-CNN ResNet	5	4	scratch	0.1604667455	0.3674731255
Faster R-CNN ResNet	6	5	tire flat	0.2012279332	0.3757142723

Hình 42: Bảng kết quả thử nghiệm Faster R-CNN

6.3. Kết quả CNN

Đối với nhánh phân loại mức độ hư hỏng, dự án sử dụng ảnh full làm đầu vào cho CNN. Ảnh người dùng upload được resize, normalize và đưa trực tiếp vào mô hình để phân loại severity tổng quát thành ba lớp minor, moderate và severe. Trong phần thực nghiệm, em đã thử nhiều kiến trúc CNN khác nhau gồm ResNet18, EfficientNet-B0, EfficientNet-B2, ResNet50 và ConvNeXt-Tiny để chọn mô hình triển khai chính.

Bảng dưới đây tổng hợp kết quả tốt nhất của các mô hình CNN đã thử nghiệm trên cùng bài toán severity classification.

Mô hình	Best validation metric	Test accuracy	Test macro F1
ResNet18	val_acc = 0.8209	0.6564	0.6490
EfficientNet-B0	val_macro_f1 = 0.7945	0.6821	0.6817
EfficientNet-B2	val_macro_f1 = 0.8022	0.6974	0.6954
ResNet50	val_acc = 0.8128	0.7026	0.7043
ConvNeXt-Tiny	val_macro_f1 = 0.8342	0.7077	0.7084

Bảng 4: So sánh kết quả các mô hình CNN cho bài toán severity classification

Kết quả cho thấy ResNet18 có validation accuracy khá cao nhưng test accuracy thấp hơn, cho thấy khả năng tổng quát hóa chưa tốt. EfficientNet-B0 và EfficientNet-B2 cải thiện so với ResNet18, đặc biệt ở chỉ số test accuracy và macro F1. ResNet50 tiếp tục cải thiện test macro F1, nhưng vẫn chưa đạt kết quả cao nhất.

ConvNeXt-Tiny đạt kết quả tốt nhất trên cả validation macro F1, test accuracy và test macro F1. Vì vậy, ConvNeXt-Tiny được chọn làm mô hình CNN chính của hệ thống, còn ResNet18 chỉ được xem là mô hình thử nghiệm/baseline ban đầu.

Để đánh giá định tính, dự án sử dụng cùng một ảnh xe bị hư hỏng và chạy inference trên các mô hình CNN đã huấn luyện. Ảnh này có nhãn đúng là MODERATE, vì vậy trong ví dụ định tính này chỉ ConvNeXt-Tiny dự đoán đúng.

Mô hình	Kết quả dự đoán	Xác suất
ResNet18	SEVERE	60.61%
EfficientNet-B0	MINOR	42.60%
EfficientNet-B2	SEVERE	42.31%
ResNet50	SEVERE	40.88%
ConvNeXt-Tiny	MODERATE	44.35%

Bảng 5: Kết quả dự đoán severity của 5 mô hình CNN trên cùng một ảnh test

Các ảnh demo dưới đây được lấy từ file PDF kết quả inference, trong đó mỗi mô hình được chạy trên cùng một ảnh đầu vào.

ResNet18

ResNet_18



Anh dau vao

Ket qua: SEVERE (60.61%)

Xac suat theo tung nhom

```
{  
  "minor" : 5.99  
  "moderate" : 33.4  
  "severe" : 60.61  
}
```


EfficientNet-B0

Phan loai



Anh dau vao

Ket qua: MINOR (42.60%)

Xac suat theo tung nhom

```
{  
  "minor" : 42.6  
  "moderate" : 36.91  
  "severe" : 20.49  
}
```

EfficientNet-B2

Phan loai



Anh dau vao

Ket qua: SEVERE (42.31%)

Xac suat theo tung nhom

```
{  
  "minor" : 39.69  
  "moderate" : 27  
  "severe" : 42.31  
}
```


ResNet50

Phan loai



Anh dau vao

Ket qua: SEVERE (40.88%)

Xac suat theo tung nhom

```
{  
  "minor" : 23.76  
  "moderate" : 35.36  
  "severe" : 40.88  
}
```

ConvNeXt-Tiny



Hình 43: So sánh kết quả dự đoán severity của 5 mô hình CNN trên cùng một ảnh test.

ConvNeXt-Tiny là mô hình duy nhất dự đoán đúng nhãn moderate trong ví dụ này.

Kết quả cho thấy các mô hình đưa ra dự đoán khác nhau trên cùng một ảnh đầu vào. ResNet18, EfficientNet-B2 và ResNet50 đều nghiêng về lớp severe, EfficientNet-B0 dự đoán minor, trong khi ConvNeXt-Tiny dự đoán moderate. Do nhãn đúng của ảnh là moderate, ConvNeXt-Tiny là mô hình duy nhất phân loại đúng trong ví dụ này. Tuy confidence của ConvNeXt-Tiny chưa quá cao, kết quả này vẫn phù hợp với bảng đánh giá định lượng, nơi ConvNeXt-Tiny đạt test accuracy và macro F1 tốt nhất. Điều này cho thấy ConvNeXt-Tiny có khả

năng cân bằng tốt hơn giữa ba lớp minor, moderate và severe so với các mô hình còn lại.

Tuy vậy, kết quả CNN vẫn chỉ nên được xem là tín hiệu hỗ trợ cho tầng rule-based adjustment. Với test accuracy khoảng 70.77%, mô hình đã đủ để minh họa vai trò của severity trong hệ thống demo, nhưng chưa nên xem là công cụ đánh giá tuyệt đối trong thực tế.

7. Đánh giá, hạn chế và hướng phát triển

7.1. Hạn chế

Mặc dù hệ thống đã xây dựng được pipeline hoàn chỉnh gồm dự đoán giá cơ bản, phát hiện hư hỏng, phân loại mức độ hư hỏng và điều chỉnh giá, dự án vẫn còn một số hạn chế nhất định.

Hạn chế đầu tiên là dữ liệu trong dự án chưa đồng bộ hoàn toàn theo từng chiếc xe. Dữ liệu bảng dùng để huấn luyện mô hình dự đoán giá xe được lấy từ một nguồn riêng, trong khi dữ liệu ảnh hư hỏng và dữ liệu phân loại mức độ hư hỏng lại đến từ các nguồn khác. Vì vậy, hệ thống hiện tại chưa học được trực tiếp mối quan hệ giữa ảnh hư hỏng của một chiếc xe cụ thể và giá bán thực tế của chính chiếc xe đó.

Hạn chế thứ hai là dự án chưa có ground truth cho giá xe sau khi xét đến hư hỏng. Mô hình tabular chỉ dự đoán giá cơ bản dựa trên các thông tin kỹ thuật của xe, còn phần điều chỉnh giá theo hư hỏng được xây dựng theo hướng rule-based. Điều này giúp hệ thống có thể hoạt động và giải thích được kết quả, nhưng mức giảm giá tham khảo vẫn phụ thuộc vào các quy tắc được thiết kế thủ công thay vì được học trực tiếp từ dữ liệu thực tế.

Hạn chế thứ ba nằm ở chất lượng và độ khó của dữ liệu ảnh hư hỏng. Một số lớp như scratch, crack và dent thường khó phát hiện hơn do vùng hư hỏng nhỏ, mảnh, dễ bị ảnh hưởng bởi ánh sáng, bóng đổ, màu sơn xe hoặc góc chụp. Điều này làm cho mô hình YOLO có thể bỏ sót hoặc nhầm lẫn giữa các loại hư hỏng có đặc điểm thị giác gần giống nhau.

Một hạn chế của hệ thống hiện tại là diện tích vùng hư hỏng được ước lượng dựa trên tỷ lệ diện tích bounding box so với toàn bộ ảnh. Cách tính này đơn giản và dễ tích hợp vào pipeline rule-based, tuy nhiên chưa phản ánh chính xác kích thước vật lý thật của hư hỏng. Với cùng một vết hư hỏng, nếu ảnh được chụp gần thì bounding box có thể chiếm tỷ lệ lớn hơn trong ảnh, làm hệ thống

đánh giá mức ảnh hưởng đến giá cao hơn. Ngược lại, nếu ảnh được chụp xa, vùng hư hỏng có thể chiếm tỷ lệ nhỏ hơn và làm mức giảm giá bị đánh giá thấp hơn.

Do đó, `area\_ratio` trong phiên bản hiện tại chỉ nên được xem là một đặc trưng tương đối trong ảnh, không phải thước đo kích thước hư hỏng thực tế. Trong các hướng phát triển tiếp theo, hệ thống có thể cải thiện bằng cách chuẩn hóa theo khoảng cách chụp, sử dụng ảnh nhiều góc, yêu cầu người dùng chụp theo hướng dẫn cố định, hoặc kết hợp thêm thông tin về vị trí bộ phận xe và kích thước tham chiếu để ước lượng diện tích hư hỏng chính xác hơn.

Hạn chế thứ tư là mô hình CNN/ConvNeXt-Tiny phân loại mức độ hư hỏng vẫn còn dư địa cải thiện. Dù ConvNeXt-Tiny là mô hình tốt nhất trong các mô hình đã thử, test accuracy khoảng 70.77% vẫn chưa đủ cao để sử dụng như một hệ thống đánh giá tuyệt đối. Lớp moderate là lớp khó vì nằm giữa minor và severe, ranh giới nhãn đôi khi mang tính chủ quan và phụ thuộc vào góc chụp, ánh sáng, mức độ rõ ràng của hư hỏng. Vì vậy, kết quả CNN nên được xem như một tín hiệu hỗ trợ cho rule-based adjustment thay vì kết luận cuối cùng về tình trạng xe.

Hạn chế thứ năm là hệ thống hiện mới dừng ở mức demo, chạy trên giao diện Streamlit và chưa được triển khai như một ứng dụng hoàn chỉnh cho người dùng thực tế. Các yếu tố như tốc độ xử lý trên nhiều ảnh lớn, khả năng lưu lịch sử định giá, quản lý người dùng, bảo mật dữ liệu và triển khai trên server chưa được xử lý đầy đủ.

7.2. Hướng phát triển

Trong tương lai, hướng phát triển quan trọng nhất là xây dựng một bộ dữ liệu đồng bộ hơn giữa thông tin xe, hình ảnh ngoại thất và giá bán thực tế. Nếu có thể thu thập dữ liệu gồm cùng một chiếc xe với đầy đủ thông tin kỹ thuật, ảnh hư hỏng và giá giao dịch sau khi xét tình trạng xe, hệ thống có thể tiến tới huấn luyện mô hình dự đoán giá cuối cùng một cách trực tiếp hơn thay vì chỉ dùng rule-based adjustment.

Một hướng phát triển tiếp theo là cải thiện chất lượng dữ liệu ảnh hư hỏng. Dự án có thể bổ sung thêm ảnh cho các lớp khó như scratch, crack và dent, đồng thời kiểm tra lại nhãn để giảm sai lệch annotation. Ngoài ra, có thể thêm các ảnh hard negative như phản sáng, vết bẩn, đường gân thân xe hoặc bóng đổ để giúp mô hình YOLO giảm nhầm lẫn trong quá trình phát hiện.

Đối với mô hình YOLO, có thể tiếp tục thử nghiệm các phiên bản mới hơn hoặc các cấu hình huấn luyện khác nhau. Ví dụ, dự án có thể so sánh thêm YOLOv8s, YOLOv8m hoặc các biến thể YOLO mới, thay đổi kích thước ảnh đầu vào, điều chỉnh augmentation, hoặc fine-tune từ checkpoint tốt nhất khi có dữ liệu mới chất lượng hơn. Tuy nhiên, việc cải thiện dữ liệu vẫn nên được ưu tiên trước khi tăng độ phức tạp của mô hình.

Đối với mô hình CNN/ConvNeXt-Tiny, có thể cải thiện bằng cách tăng số lượng ảnh huấn luyện, cân bằng lại số mẫu giữa các lớp minor, moderate và severe, áp dụng data augmentation hợp lý và đánh giá thêm bằng confusion matrix. Ngoài ConvNeXt-Tiny, dự án cũng có thể thử các kiến trúc khác như ResNet50, EfficientNet hoặc MobileNet để so sánh giữa độ chính xác và tốc độ suy luận.

Phần điều chỉnh giá cũng có thể được cải tiến trong tương lai. Hiện tại, mức giảm giá được tính theo luật dựa trên loại hư hỏng, diện tích vùng hỏng và severity tổng quát. Khi có dữ liệu thực tế hơn, có thể thay thế hoặc hiệu chỉnh tầng rule-based bằng một mô hình học máy, trong đó các đặc trưng từ YOLO và CNN được đưa vào cùng với dữ liệu bảng để dự đoán mức giảm giá hoặc giá tham khảo cuối cùng một cách trực tiếp hơn.

Về mặt hệ thống, ứng dụng có thể được mở rộng từ demo Streamlit thành một web application hoàn chỉnh hơn. Các chức năng có thể bổ sung gồm lưu lịch sử định giá, xuất báo cáo PDF, cho phép người dùng tải nhiều ảnh theo từng góc chụp, quản lý thông tin xe đã định giá và triển khai lên server để người dùng có thể truy cập từ xa.

Cuối cùng, nếu hướng đến ứng dụng thực tế tại Việt Nam, dự án cần bổ sung dữ liệu xe cũ từ thị trường Việt Nam. Dataset hiện tại chủ yếu phản ánh thị trường nước ngoài, nên giá trị dự đoán có thể chưa sát với giá xe tại Việt Nam. Việc thu thập dữ liệu từ các trang mua bán xe trong nước, kết hợp với tỷ giá, thương hiệu phổ biến và điều kiện sử dụng thực tế sẽ giúp hệ thống định giá phù hợp hơn với bối cảnh ứng dụng trong nước.

8. Kết luận

Trong dự án này, em đã xây dựng một hệ thống demo hỗ trợ dự đoán giá xe ô tô cũ có xét đến tình trạng hư hỏng ngoại thất bằng cách kết hợp Machine Learning và Computer Vision. Pipeline chính của hệ thống gồm XGBoost cho

dự đoán giá từ dữ liệu bảng, YOLOv8s trên merge_Data cho phát hiện hư hỏng, ConvNeXt-Tiny cho phân loại mức độ hư hỏng từ ảnh full và tầng rule-based adjustment để tính giá sau điều chỉnh.

Về phần dự đoán giá cơ bản, dự án đã thực hiện các bước thu thập dữ liệu, làm sạch dữ liệu, xử lý giá trị thiếu, mã hóa biến phân loại, tạo đặc trưng mới và huấn luyện các mô hình hồi quy như Linear Regression, Random Forest và XGBoost. Kết quả thực nghiệm cho thấy XGBoost là mô hình chính cho tabular price prediction vì đạt hiệu quả tốt nhất trong nhóm mô hình so sánh.

Về phần xử lý ảnh, YOLOv8s trên merge_Data được chọn làm mô hình chính cho damage detection. Mô hình này giúp hệ thống xác định vị trí hư hỏng, loại hư hỏng, độ tin cậy và tỷ lệ diện tích vùng hỏng. Song song với đó, ConvNeXt-Tiny được chọn làm mô hình chính cho severity classification từ ảnh full, phân loại mức độ hư hỏng tổng quát thành minor, moderate hoặc severe.

Một điểm quan trọng của dự án là việc thiết kế pipeline nhiều mô-đun, trong đó mỗi thành phần đảm nhiệm một nhiệm vụ riêng: XGBoost dự đoán base price, YOLOv8s tạo damage features, ConvNeXt-Tiny tạo severity signal và tầng rule-based adjustment tính toán final adjusted price. Cách tiếp cận này giúp hệ thống dễ hiểu, dễ mở rộng và phù hợp với phạm vi của một dự án thực nghiệm.

Thông qua giao diện demo bằng Streamlit, người dùng có thể nhập thông tin xe và nhận giá cơ bản từ XGBoost. Nếu người dùng không upload ảnh, hệ thống chỉ dùng dữ liệu bảng và final price bằng base price. Nếu người dùng upload ảnh, hệ thống mở rộng phân tích hư hỏng bằng hai nhánh YOLOv8s và ConvNeXt-Tiny, sau đó hiển thị danh sách hư hỏng, mức độ severity và giá xe tham khảo sau điều chỉnh.

Tuy nhiên, dự án vẫn còn một số hạn chế. Các bộ dữ liệu sử dụng trong hệ thống chưa đồng bộ hoàn toàn theo từng chiếc xe, chưa có nhãn giá bán sau khi xét đến hư hỏng, và phần điều chỉnh giá hiện vẫn dựa trên luật thay vì được học trực tiếp từ dữ liệu thực tế. Ngoài ra, mô hình phát hiện và phân loại hư hỏng vẫn có thể được cải thiện thêm, đặc biệt với các lớp khó như vết xước, vết nứt, vết móp và lớp severity moderate.

Nhìn chung, dự án đã hoàn thành được mục tiêu xây dựng một hệ thống thử nghiệm kết hợp dữ liệu bảng và dữ liệu ảnh trong bài toán định giá xe ô tô cũ. Qua quá trình thực hiện, em đã có cơ hội vận dụng các kiến thức về xử lý dữ liệu, Machine Learning, Deep Learning, Computer Vision và xây dựng ứng dụng demo. Đây là nền tảng quan trọng để tiếp tục phát triển hệ thống trong

tương lai, đặc biệt theo hướng bổ sung dữ liệu thực tế tại thị trường Việt Nam, cải thiện mô hình hư hỏng và xây dựng cơ chế định giá sát thực tế hơn.

9. Tài liệu tham khảo

Pytorch và python:

- https://www.learnpytorch.io/03_pytorch_computer_vision/
- <https://www.w3schools.com/>

Video, tài liệu về xgboost: <https://www.youtube.com/watch?v=a4jzMCXkQ0U>

Tài liệu, tìm hiểu về Decision Tree:

<https://machinelearningcoban.com/2018/01/14/id3/>

CNN và ConvNeXt-Tiny:

- https://vi.d2l.ai/chapter_convolutional-modern/index.html
- <https://cnn-playground.live/>

YOLO và các phiên bản: <https://docs.ultralytics.com/models/yolov8>

Metrics:

- <https://docs.ultralytics.com/reference/utils/metrics>

Faster-RCNN:

- <https://www.ultralytics.com/blog/what-is-r-cnn-a-quick-overview>
- <https://minhtrieu.vn/thi-giac-may-machine-vision/phat-hien-doi-tuong-voi-yolo-faster-r-cnn/>

Finetune và Transfer learning:

https://docs.pytorch.org/tutorials/beginner/transfer_learning_tutorial.html

Object detection: <https://docs.ultralytics.com/tasks/detect>

Dataset sử dụng trong dự án:

<https://www.kaggle.com/datasets/asfarhossainsitab/car-damage-detection>

<https://www.kaggle.com/datasets/avikasliwal/used-cars-price-prediction>

<https://universe.roboflow.com/autodentify/car-damage-detection-ggmju>

<https://universe.roboflow.com/nibm-7v215/scratch-and-dent-xvjy5>